

# Från kundorder till verifierad roboteffekt

Vetenskaplig designstudie för RobotOps Twin

Rami Halabi

Version 1.0 · 1 oktober 2026

Oberoende tekniskt rapportpaket · Svenska

**Status: design och forskningsunderlag.** Simulatoren beskrivs som ett planerat system. Inga empiriska robotresultat eller säkerhetsgarantier hävdas.

AI-assisterad text- och publiceringsframställning.

Ej sakkunniggranskad. Inte framtagen av eller godkänd av SICS AI.

Rapporternas källor, diagram och byggsript finns i projektets repository.

# Innehåll

|                                                             |    |
|-------------------------------------------------------------|----|
| Sammanfattning.....                                         | 3  |
| 1. Syfte, frågeställningar och läsanvisning .....           | 3  |
| 2. Metod och källkritik .....                               | 5  |
| 3. Vad de offentliga källorna faktiskt visar .....          | 6  |
| 4. Krav och minsta meningsfulla omfattning .....            | 8  |
| 5. Föreslagen systemarkitektur.....                         | 9  |
| 6. Datakontrakt och tillstånd .....                         | 11 |
| 7. Kärnfallet: utfört plock, förlorad kvittens .....        | 14 |
| 8. Observation, modell och verifiering.....                 | 16 |
| 9. Blender, MCP och operationella gränser .....             | 18 |
| 10. Utvärderingsplan: hur vi avser att testa designen ..... | 20 |
| 11. Genomförande, dokumentation och underhåll .....         | 22 |
| 12. Diskussion, begränsningar och slutsats .....            | 23 |
| Referenser och källbegränsningar .....                      | 23 |

## Sammanfattning

RobotOps Twin är en föreslagen, lokal integrationssimulator för lagerrobotik. Ett förenklat ERP/WMS skapar plockuppdrag. Ett Pythonbaserat integrationslager validerar och lagrar uppdragen, hämtar observationer, begär ett handlingsförslag och skickar typade kommandon till en simulerad robotcell i Blender. En separat verifierare bedömer utfallet innan kundsystemet uppdateras. Den centrala frågan är inte hur en robotarm animeras, utan hur mjukvaran hanterar skillnaden mellan en begärd handling, en kvitterad operation och ett observerat resultat.

Studien kombinerar en riktad genomgång av offentliga primärkällor med ett eget arkitekturförslag och en förhandsdefinierad utvärderingsplan. Underlaget omfattar SICS AI:s rollbeskrivning och HYPER-redovisning, Cognibotics produktionsbeskrivning, CloudGripper/AutoGrasper, R900 samt officiell dokumentation för Blender, OpenAI och integrationsprotokoll. Källorna har olika bevisvärde och hålls därför isär.

**Resultatet i denna version är en dokumenterad design, inte en implementerad eller experimentellt validerad robotlösning.** Inga prestandasiffror för projektet, säkerhetsgarantier eller resultat om generell intelligens hävdas. Det planerade bidraget är ett litet, reproducerbart testsystem där fel efter en fysiskliknande sidoeffekt kan studeras utan verklig robotutrustning.

**Nyckelord:** systemintegration, lagerrobotik, Blender, verifiering, idempotens, återhämtning, observerbarhet, digital-tvilling-inspirerad simulering.

---

## 1. Syfte, frågeställningar och läsanvisning

Projektet ska visa hur ett kunduppdrag kan följas genom flera system med olika ansvar och olika uppfattningar om vad som har inträffat. Den avsedda användningen är teknisk diskussion, intervjuförberedelse och ett senare portfolioprojekt. Det är inte en beställning från SICS AI och inte dokumentation av företagets interna implementation.

Tre frågor styr designen:

**F1.** Hur kan ett litet system hålla isär affärsuppdrag, modellförslag, robotkommandon och verifierade effekter?

**F2.** Hur kan systemet återhämta sig när kommunikationen försvinner efter att ett objekt redan har flyttats, utan ett omotiverat nytt plock?

**F3.** Vilka relevanta integrationsproblem kan demonstreras i Blender, och vilka slutsatser kräver fysisk utrustning eller tillgång till den verkliga plattformen?

Rapport 1 beskriver forskningsunderlag, arkitektur och testplan. Rapport 2 granskar överförbarheten till en verklig robotmiljö. Rapport 3 är ett samtalsunderlag för diskussion med Axel Kaliff och Per-Eric Olsson. Den som vill förstå huvudidén först kan läsa figur 1, avsnitt 7 och slutsatsen. Kontrakt och utvärderingsprotokoll är till för den tekniska fördjupningen.

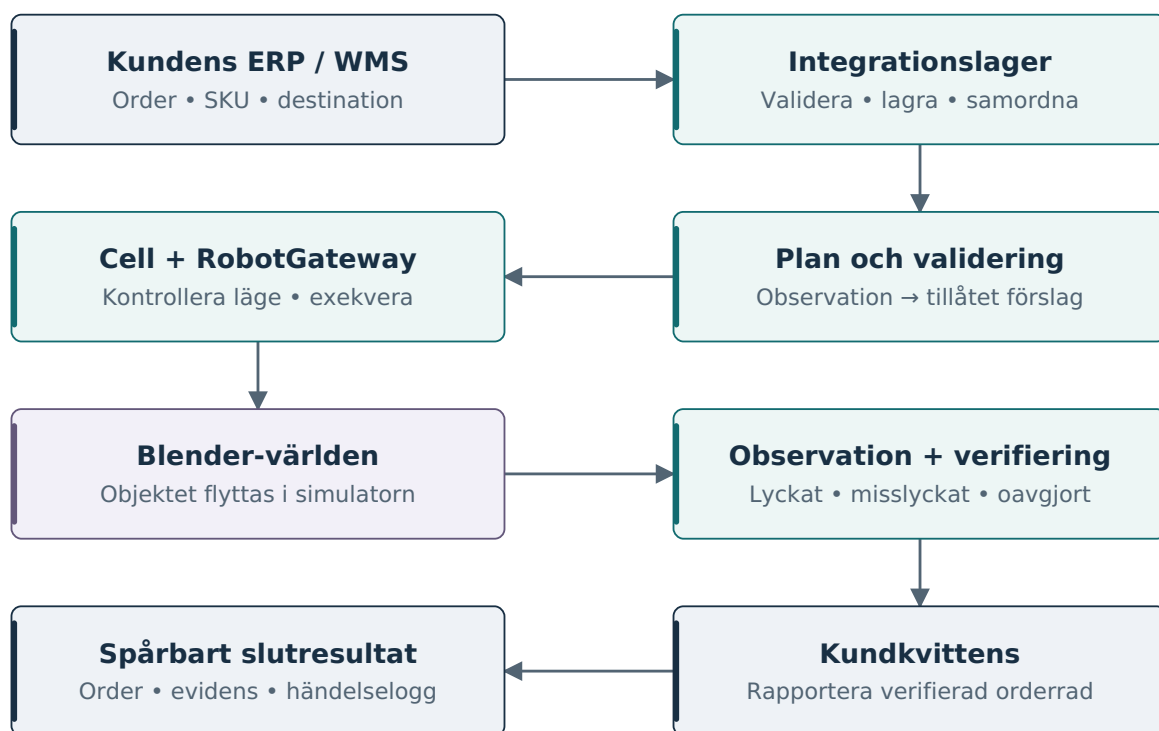
### 1.1 Vad ordet "Twin" betyder här

Projektets namn är RobotOps Twin. I denna studie används dock den mer precisa beskrivningen **digital-tvilling-inspirerad integrationssimulator**. Det finns ännu ingen ansluten fysisk tillgång som modellen hålls synkroniserad med. En synlig Blender-scen är därför inte i sig en verifierad digital tvilling av Nowastes anläggning.

Vi definierar en *effekt* som en förändring i simulatorns auktoritativa världstillstånd, exempelvis att objekt O17 flyttas från källåda A till orderlåda B. Att en animation har spelats upp är inte en tillräcklig effektdefinition. Testerna måste kunna inspektera den lagrade världen och effektjournalen.

#### Från kundorder till verifierad effekt

Föreslaget flöde • en cell • ett aktivt plock



Figur 1. Egen konceptmodell. Ett plock passerar flera ansvar; kundstatus ändras först efter observerad effekt och hanterad kvittens. Pilarna visar informations- och exekveringsflöde, inte verkliga SICS-gränssnitt.

## 2. Metod och källkritik

### 2.1 Undersökningens karaktär

Detta är en **riktad litteratur- och dokumentstudie med konstruktivt designförslag**. Det är inte en uttömmande systematisk översikt. Urvalet utgår från den aktuella rollen, de namngivna forskarna och de produkter som förekommer i offentliga integrationsbeskrivningar. Sökningen har kompletterats med officiell teknisk dokumentation för föreslagna verktyg.

Kontrolldatum är **1 oktober 2026**. Referensregistret anger organisation eller författare, titel, datum när det är känt, URL och begränsning. Källor utan säkert publiceringsdatum markeras utan datum. En relativ LinkedIn-tidsstämpel omvandlas inte till ett gissat publiceringsdatum.

Arbetsgången var att identifiera relevanta påståenden, kontrollera deras ursprung, härleda ett integrationsbehov och formulera ett eget designbeslut. Författarens beslut redovisas som förslag, inte som fynd om SICS AI. Tidigare research har behandlats som ett sökunderlag och inte som en oberoende källa.

### 2.2 Fyra evidens kategorier

| Kategori           | Betydelse i rapportpaketet                                               | Exempel                                              |
|--------------------|--------------------------------------------------------------------------|------------------------------------------------------|
| Dokumenterat       | En identifierbar källa innehåller den beskrivna uppgiften.               | En rollannons nämner Pythonintegration.              |
| Självrapporterat   | Företaget eller projektgruppen beskriver egna egenskaper eller resultat. | HYPER-redovisning och leverantörspress.              |
| Eget designförslag | Ett val för RobotOps Twin, härlett men inte kopierat från källorna.      | SQLite, kommandokontrakt och testfall.               |
| Ej verifierat      | Underlag saknas, är otillgängligt eller otillräckligt för slutsatsen.    | SICS interna API, säkerhetsarkitektur och modellkod. |

En EU-webbplats kan verifiera att ett projekt och en rapport finns. Det gör inte automatiskt deltagarens tekniska resultat oberoende reproducerade. En tillverkares högsta uppgivna plockhastighet ska inte tolkas som mätt genomströmning hos en viss kund.

Axels examensarbete används med särskild försiktighet: titel och bibliografiskt sammanhang identifierades i föregående research, men fulltexten kunde inte återhämtas i denna publiceringskontroll. Därför återges inga detaljerade metod- eller resultatpåståenden från examensarbetet. Kopplingen till verifiering används som ett avgränsat diskussionstema. **[S04]**

### 2.3 Publiceringsetik och reproducerbarhet

Alla figurer och arkitekturkontrakt i paketet är egna illustrationer. Inga företagslogotyper, privata e-postmeddelanden eller interna dokument återpubliceras. Referenserna är inspiration och belägg för avgränsade påståenden, inte ett påstående om samarbete eller godkännande.

Text, disposition och publiceringskod har tagits fram med AI-assistans. Projektägaren ansvarar för att läsa och förstå materialet före användning. Rapportpaketet är inte sakkunniggranskat. Källfiler, diagramdefinitioner och byggsript publiceras så att text och figurer går att rätta och regenerera.

### 3. Vad de offentliga källorna faktiskt visar

#### 3.1 Rollen: integrationsansvar snarare än enbart modellutveckling

Den granskade Systemingenjör-annonsen kopplar rollen till Python, robotceller och kundernas API:er, PLC:er och WMS/ERP. Den omfattar också LLM/VLM-integration, driftsättning, felsökning, dokumentation och teknisk kunddialog. Industriella protokoll anges som meriterande. Annonsen är ett kravunderlag, inte en fullständig lista över den interna teknikstacken. [S01]

En närliggande annonsversion formulerar erfarenhetskraven annorlunda. Projektet ska därför inte användas för att hävda att alla formella anställningskrav är uppfyllda. Det demonstrerar valda arbetsuppgifter; det ersätter inte yrkeserfarenhet, examen eller ansvar för verkliga leveranser. [S02]

#### 3.2 SICS AI:s HYPER-beskrivning

HYPER är registrerat hos CORDIS med projekt-ID 101218436. I deltagarrapporteringen beskriver SICS AI en strukturerad världsmodell och feedbackbaserat lärande samt anger att kärnan inte är en LLM och inte använder reinforcement learning. Rapporten beskriver också robotgränssnitt med kontroll på led- och rörelsenivå. Detta är bolagets redovisning; underlaget räcker inte för att rekonstruera algoritmen. [S03]

För projektet följer ett designkrav: AI-funktionen ska vara en utbytbar komponent med ett uttryckligt in- och utkontrakt. Den ska inte ersättas med en fri textprompt som samtidigt får obegränsad rätt att ändra simulatorn.

#### 3.3 Produktionsbeskrivningen från Cognibotics

Cognibotics beskriver den 5 januari 2026 en andra robotcell för Nowaste med AutoStore, SICS AI:s vision och integration mot lagerstyrningen. Juliet & Romeo tillskrivs realtidsrörelse, gripning och överlämning till transportband. Det är en leverantörsbeskrivning av en konkret installation, inte en publicerad intern gränssnittsspecifikation. [S07]

Tillverkaren beskriver HKM1800 som hybridkinematisk. RobotOps Twin ska därför inte märka en generisk sexaxlig Blender-arm som en mekaniskt korrekt HKM1800. Robotens utseende används för förståelse av flödet, inte för att härleda dess faktiska arbetsområde eller cykeltid. [S08]

HYPER- och produktionsbeskrivningarna behöver inte motsäga varandra. De kan avse olika nivåer, versioner eller tillämpningar. Vårt svar är att skilja mellan uppgiftsnivå och rörelsenivå i designen och be SICS förklara den verkliga ansvarsfördelningen. Två fullständiga styrimplementationer behövs inte i första versionen.

### 3.4 Axel Kaliff, AutoGrasper och R900

CloudGripper/AutoGrasper beskriver ett arbetsflöde där uppgift, återställning, felåterhämtning och datainsamling har separata roller. README-filen anger tillstånden STARTUP, ACTIVE och RESETTING och namnger metoder för normal uppgift respektive återhämtning. Det granskade README-objektets Git-blob är 3cb1da256bc5147ef52a56327da476f110c4e6c2. Det är forskningsverktyg, inte SICS produktkod. [S05]

R900, med Axel Kaliff som medförfattare, undersöker kostnadseffektiviteten hos autonom datainsamling för robotinlärning. Den granskade arXiv-posten har en reviderad version från 17 september 2025. Studiens automatisering av försök, märkning och återställning motiverar att också vår simulator gör episoder och återhämtning till förstklassiga begrepp. Vi reproducerar inte dess dataset eller modellresultat. [S06]

Axels examensarbetstitel anknyter till verifiering av robotaktioner med en digital tvilling. Denna tematiska koppling är relevant, men innebär inte att en efterhandskontroll av objektposition i Blender reproducerar hans forskningsmetod. [S04]

### 3.5 GPU-resurser och deras roll

ENCCS redovisar arbete med Python/PyTorch och GPU-träning på LUMI. Aixia har offentliggjort en leverans av DGX B200 med drifttjänster till SICS AI. Dessa källor ger stöd för verkligt beräkningsarbete, men fastställer inte var varje kunds inferens körs eller vilket GPU-ansvar den aktuella rollen får. [S10] [S11]

Simulatorn behöver ingen B200. Den ska i stället kunna beskriva modellversion, inferensgränssnitt, latens och felutfall. En eventuell framtida GPU-adapter ändrar inte vem som äger ordern eller vem som verifierar ett plock.

### 3.6 Från inspiration till eget beslut

| Källa                     | Avgränsat fynd                              | Eget designbeslut                              | Inte en reproduktion av               |
|---------------------------|---------------------------------------------|------------------------------------------------|---------------------------------------|
| Rollannons [S01]          | Integration mellan flera system.            | Separata kund-, modell- och robotkontrakt.     | Företagets exakta stack.              |
| HYPER [S03]               | Världsrepresentation och robotgränssnitt.   | Ett typat Brain-gränssnitt.                    | Proprietär modell eller AGI.          |
| AutoGrasper [S05]         | Uppgift, reset och recovery skiljs åt.      | Episodmodell och explicita felvägar.           | Deras implementation eller data.      |
| R900 [S06]                | Automatiserade robotförsök.                 | Reproducerbara scenarier och loggning.         | Studiens resultat.                    |
| Cognibotics [S07]         | Motion, vision och lagerstyrning samverkar. | Separat cell- och robotadapter.                | Den installerade cellens interna API. |
| Blender [S12] [S13] [S14] | Scen-API, timers och MCP-verktyg.           | Kontrollerad runtime, separat scenförfattande. | Industriell realtidsstyrning.         |

| Källa     | Avgränsat fynd                                   | Eget designbeslut                                | Inte en reproduktion av         |
|-----------|--------------------------------------------------|--------------------------------------------------|---------------------------------|
| AWS [S17] | Återförsök kräver tydlig identitet och semantik. | Beständig avsikt och avstämning av okänt utfall. | En allmän exactly-once-garanti. |

## 4. Krav och minsta meningsfulla omfattning

### 4.1 Ett litet system med verkliga ansvarsskillnader

Första versionen ska ha en kund, en robotcell, tre syntetiska artikeltyper och ett aktivt plock åt gången. En orderrad omfattar ett exemplar; större kvantiteter delas senare upp i spårbara deluppdrag. Källlåda, destinationslåda och produktidentiteter är explicit modellerade.

Systemet ska kunna köras utan externa modellkonton. Blender visar förändringen i världen, men ordern blir inte färdig bara för att animationen slutar. UI:t visar orderstatus, cellstatus, kommandostatus och verifieringsstatus var för sig.

### 4.2 Kravspårbarhet mot rollen

| ID  | Arbetsområde     | Planerad projektartefakt                       | Vad som kan bedömas                              |
|-----|------------------|------------------------------------------------|--------------------------------------------------|
| K01 | Pythonutveckling | Modulärt backend och typade modeller.          | Kodstruktur, felhantering och tester.            |
| K02 | Kund-API/WMS     | Mock-ERP och adapter med kvittenser.           | Kontrakt och affärssemantik.                     |
| K03 | Robotintegration | RobotGateway med Blender-adapter.              | Ansvarsgräns och versionshantering.              |
| K04 | PLC/cellsignaler | Logisk CellController.                         | Interlocks och tillstånd, inte PLC-certifiering. |
| K05 | LLM/VLM          | Valfri adapter för strukturerade förslag.      | Validering, tidsgränser och avvisning.           |
| K06 | Driftsättning    | Lokal installationsguide och versionsmanifest. | Repeterbar start och konfiguration.              |
| K07 | Drift/felsökning | Händelsetidslinje och felinjektion.            | Spårbarhet och diagnos.                          |
| K08 | Tillförlitlighet | Journal, dubblettskydd och reconciliation.     | Modellens definierade felantaganden.             |
| K09 | Prestanda        | Mätning per steg och kötid.                    | Flaskhalsanalys, inte industriell kapacitet.     |

| ID  | Arbetsområde                 | Planerad projektartefakt                                | Vad som kan bedömas                                        |
|-----|------------------------------|---------------------------------------------------------|------------------------------------------------------------|
| K10 | Dokumentation/<br>kunddialog | Tre rapporter, kontrakt<br>och<br>demonstrationsmanus.  | Förmåga att förklara<br>beslut.                            |
| K11 | Industriella protokoll       | OPC UA-adapter som<br>tillägg med kontraktstest.        | Protokollintegration först<br>när den är<br>implementerad. |
| K12 | AI-stödd utveckling          | Versionshanterade<br>ändringar och granskade<br>tester. | Eget ansvar för<br>agentgenererad kod.                     |

K01-K10 är förberedelsemål, inte genomförda leveranser i denna rapportversion. K11 får inte markeras färdigt för att booleska signaler finns i JSON. Annonsens kärnuppgifter och meriterande områden är utgångspunkt för kartläggningen. [\[S01\]](#) [\[S02\]](#)

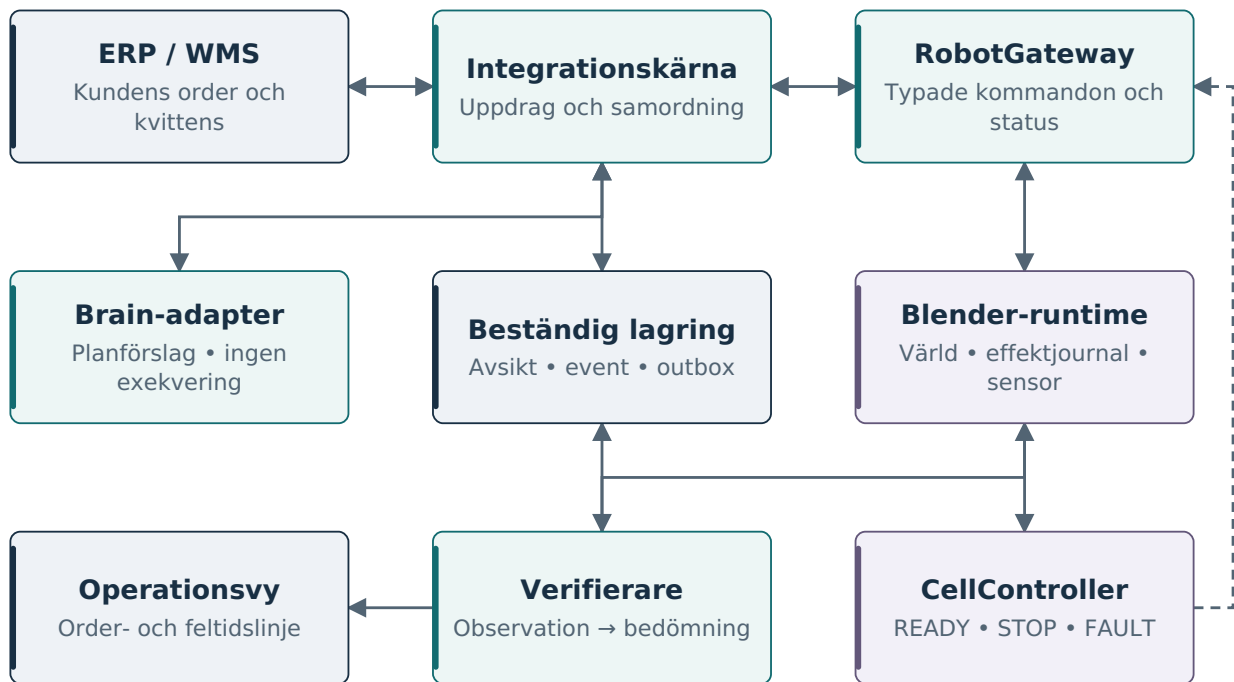
## 5. Föreslagen systemarkitektur

### 5.1 Komponenter och ägarskap

Arkitekturen är en liten modulär tjänst med separata externa gränser, inte ett stort kluster av mikrotjänster. Ett separat mock-kundsystem och en separat Blenderprocess gör kommunikationsfel möjliga att studera. Modell, verifierare och cellkontroll kan inledningsvis vara moduler i samma Pythonapplikation.

## Systemgränser och ägarskap

Moduler är inte automatiskt separata mikrotjänster



Figur 2. Egen komponentarkitektur. Integrationskärnan äger uppdraget; Blender äger simulatortillståndet. Verifiering lagras och återkopplas till kunden. Dubbelriktade pilar anger begäran/svar eller läsning/återkoppling, inte gemensamma transaktioner. Celltillåtelse är här en logisk emulering.

| Del               | Äger                                            | Får inte göra                                         |
|-------------------|-------------------------------------------------|-------------------------------------------------------|
| Mock ERP/WMS      | Order, artikelregister och kundens kvittens.    | Skriva robotleder eller modellens interna tillstånd.  |
| Integrationslager | Uppdrag, avsikt, försök och rapportering.       | Tolka timeout som bevis för utebliven effekt.         |
| Brain-adapter     | Ett handlingsförslag och modellmetadata.        | Exekvera kod i Blender eller godkänna sig själv.      |
| Validator         | Kontrakt, aktualitet och operationella gränser. | Påstå att logiska kontroller är funktionell säkerhet. |
| CellController    | Simulerad tillgänglighet, stopp och signaler.   | Återstarta rörelse automatiskt efter reset.           |
| RobotGateway      | Kommandostatus och adapterförmågor.             | Dölja skillnader mellan stödda och ostödda kommandon. |
| Blender-runtime   | Simulatorns värld och effektjournal.            | Låta UI eller modell manipulera världen direkt.       |

| Del         | Äger                                            | Får inte göra                                          |
|-------------|-------------------------------------------------|--------------------------------------------------------|
| Verifierare | Bedömning baserad på observationer.             | Anta att sänd kvittens innebär lyckad uppgift.         |
| Test-orakel | Jämförelse med fullständigt simulatortillstånd. | Ge den ordinarie beslutsvägen obegränsad facitåtkomst. |

## 5.2 Två arkitekturnivåer, en första implementation

Det logiska gränssnittet kan uttrycka både `PickIntent` och `JointTrajectory`, men första implementationen ska endast utföra en avgränsad `PICK_AND_PLACE`-uppgift via fördefinierade, validerade rörelsesegment. En kapabilitetsförfrågan anger vad adaptorn faktiskt stöder. En ostödd ledtrajectoria ger ett tydligt fel, inte en approximativ animation som låtsas vara motsvarande styrning.

Senare kan en lednivåadapter införas med lednamn, tidsatta waypoints och explicit robotmodell. Det är ett separat integrationsarbete med egna tester. Gemensamt affärskontrakt kan minska ändringar i ERP-delen, men gör inte ett hårdvarubyte plug-and-play.

## 5.3 Driftmiljö

Den avsedda körningen är lokal: webbläsare, Pythonprocesser, SQLite och Blender. Kundmocken bör ha en separat databas eller åtminstone ett separat lagringskontrakt så att kundkvittens inte blir en dold lokal transaktion. Ett processövervakningsskript kan starta komponenterna. Containerisering av backend är valfri; Blender kan köras direkt på värddatorn.

GitHub Pages används endast för rapporter, diagram och nedladdningar. Pages kör inte Pythonbackend eller Blender. En framtida inspelad demo ska märkas inspelad och inte presenteras som levande robottelemetri. [\[S19\]](#)

# 6. Datakontrakt och tillstånd

## 6.1 Identiteter som inte får blandas ihop

`order_id` identifierar kundens order. `job_id` identifierar den affärseffekt som efterfrågas. `command_id` identifierar ett specifikt exekveringsförsök. `event_id` används för mottagardubbletter. `scene_epoch` identifierar en sammanhängande simulatorvärld och ändras vid återställning av scenen. En scenåterställning får inte radera tidigare orderhistorik eller få gamla kommandon att bli giltiga igen.

Återleverans av samma orderbegäran använder samma idempotensnyckel. Samma nyckel med annat innehåll avisas som konflikt. Ett nytt försök efter ett verkligt misslyckande får egen försöksidentitet men behåller kopplingen till samma uppdrag. Skillnaden mellan begäran och avsikt är central i designen. [\[S17\]](#)

## 6.2 Föreslaget kommando

Nedanstående JSON är ett designexempel, inte SICS AI:s API:

```
{
  "schema_version": "1.0",
  "command_id": "CMD-017-A1",
  "job_id": "JOB-017",
  "scene_epoch": "SCENE-004",
  "kind": "PICK_AND_PLACE",
  "object_id": "OBJ-RED-01",
  "source_id": "SOURCE-A",
  "destination_id": "ORDER-TOTE-17",
  "observation_id": "OBS-104",
  "calibration_version": "CAL-001",
  "frame_id": "cell_world",
  "target_pose": {
    "position_m": [0.42, -0.18, 0.12],
    "quaternion_xyzw": [0.0, 0.0, 0.0, 1.0]
  },
  "planner_version": "deterministic-1"
}
```

Valideringen kontrollerar bland annat tillåten kommandotyp, ändliga tal, rätt koordinatsystem, korrekt objekt, rimliga gränser, kompatibel kalibrering och tillräckligt aktuell observation. Kvaternionen kontrolleras mot en dokumenterad normtolerans. Toleranserna är egna simulatorparametrar och får inte kallas industriellt säkra.

## 6.3 Föreslagna API-gränser

| Gräns                 | Exempel                             | Semantik                                               |
|-----------------------|-------------------------------------|--------------------------------------------------------|
| Kund till integration | POST /v1/pick-jobs                  | Skapa eller återfå samma uppdrag via idempotensnyckel. |
| Uppföljning           | GET /v1/jobs/{job_id}               | Affärsstatus och länkar till försök/evidens.           |
| Modell                | POST /v1/plans                      | Förslag, inte exekvering.                              |
| Robot                 | POST /v1/commands                   | Accepterad betyder inte utförd.                        |
| Robotstatus           | GET /v1/commands/{command_id}       | Journalens status inom angiven scene_epoch.            |
| Observation           | GET /v1/cells/{cell_id}/observation | Tidsatt observation med sensorursprung.                |
| Kundkvittens          | POST /v1/order-events               | Dubbeltskyddad uppdatering av orderrad.                |
| Felscenarier          | POST /test/faults                   | Endast lokal testmiljö; inte produktionsgränssnitt.    |

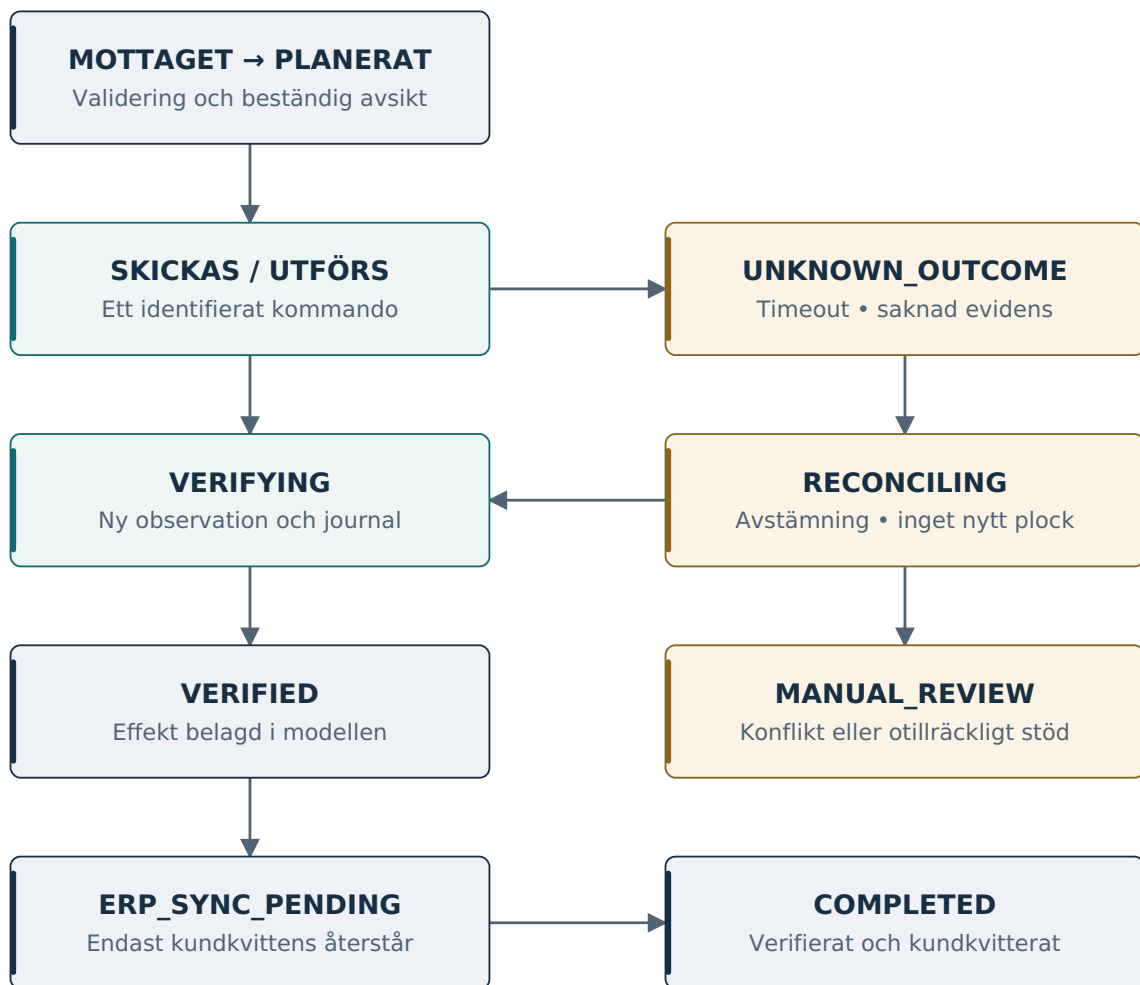
API-kontrakten ska dokumentera timeout, felkod, tidsstämpel, behörighet och vilka fält som ingår i innehållshashen. Felkod 404 från robotjournalen bevisar inte i sig att kommandot aldrig kördes: journalens beständighet, epok och retention måste också vara kända.

## 6.4 Uppdragstillstånd och celltillstånd

Uppdraget beskriver vad affärsflödet vet. Celltillståndet beskriver om simulatoren kan fortsätta köra. Dessa är separata tillståndsmaskiner. En robot kan ha avslutat en rörelse medan uppdraget fortfarande har okänt utfall.

### Uppdragets tillstånd och osäkra utfall

Förenklad vy • cellens tillståndsmaskin är separat



Figur 3. Egen tillståndsmodell. Mottagning och planering sammanfattas för läsbarhet. UNKNOWN\_OUTCOME är kunskapsbrist, inte bevis för ett misslyckat plock. Reconciliation kan gå vidare till verifiering eller kräva granskning. ERP-fel återförsöker bara synkroniseringen.

Normalvägen är mottaget, validerat, kölagt, planerat, skickas, utförs, verifieras, verifierat och kundsynk väntar. COMPLETED tillåts först när kundsystemets kvittens är hanterad. Om endast kundsynk misslyckas ska roboten inte plocka igen.

UNKNOWN\_OUTCOME betyder att underlaget inte räcker. Reconciliation är avstämning, inte ännu ett robotkommando. Om bevisen fortfarande är otillräckliga går ärendet till MANUAL\_REVIEW eller kvarstår som oavgjort. Det får inte automatiskt bli FAILED enbart för att en tidsgräns passerat.

Cellen kan vara BOOTING, READY, RUNNING, FAULTED, STOPPED eller RESETTING. Ett återställt stopp återställer inte automatiskt planens giltighet. En ny observation och ny tillåtelse behövs innan en avbruten uppgift återupptas.

---

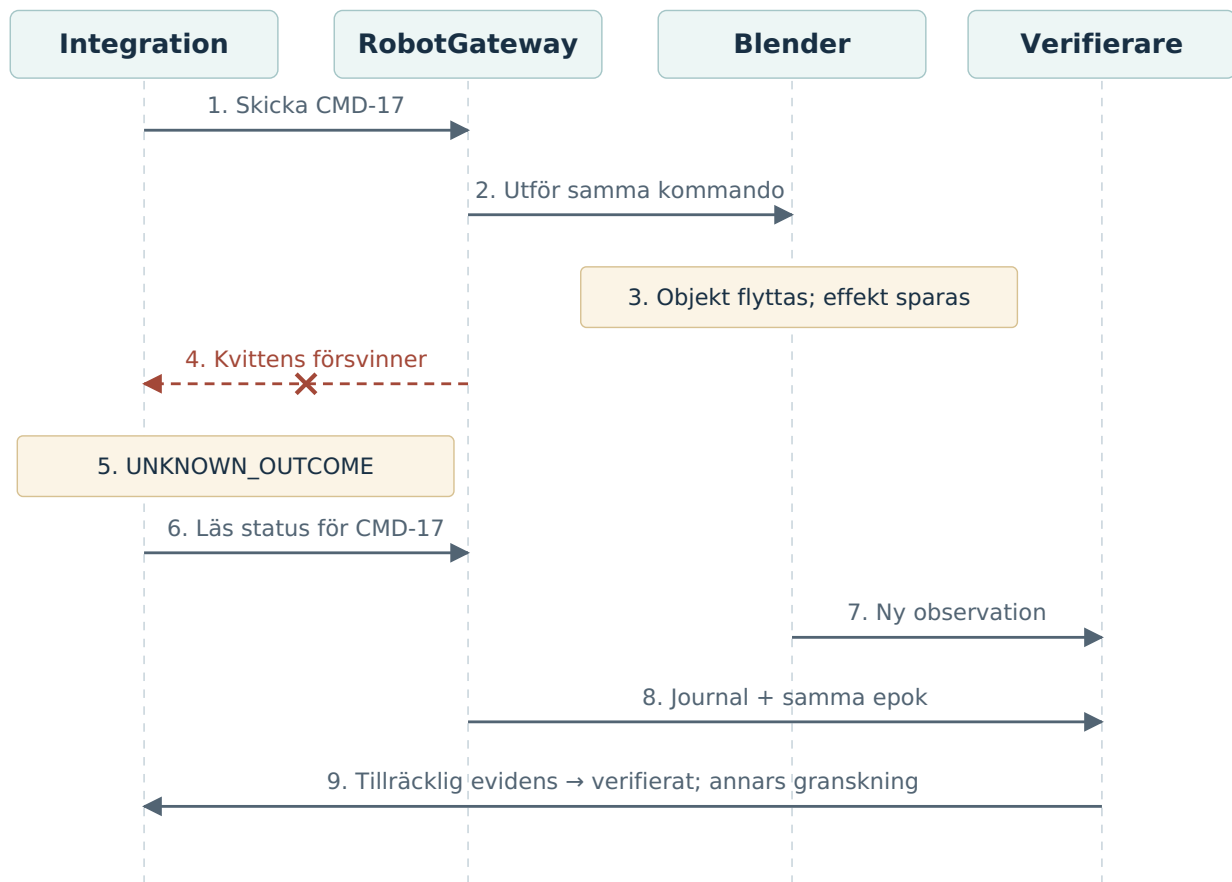
## 7. Kärnfallet: utfört plock, förlorad kvittens

### 7.1 Varför ett återförsök kan vara fel

Anta att robotadaptern flyttar ett objekt, men att nätverket bryts innan integrationstjänsten får svaret. Klienten ser bara en timeout. Två förlopp är då förenliga med samma observation: ingenting utfördes, eller allt utfördes men svaret försvann. Utan mer information går det inte att välja ett säkert nytt plock enbart utifrån timeouten.

## Utfört plock — förlorad kvittens

Planerat felscenario T04 • inget empiriskt testresultat



**Ingen ny PICK skickas under avstämningen.**

Figur 4. Egen sekvensmodell. Effekten uppstår före kommunikationsfelet. Integrationen begär kommandostatus och observation, inte ett nytt plock. Endast tillräcklig och konsistent evidens tillåter slutförande; annars krävs fortsatt utredning.

Den föreslagna lösningen kombinerar beständig avsikt, kommandoregister och observation efter handling. Den utlovar inte en universell exactly-once-egenskap. Ett databaslås och en robotrörelse är inte en gemensam atomär transaktion.

## 7.2 Föreslaget återhämtningsförlopp

Integrationslagret lagrar avsikten före sändning. Blender-adaptern lagrar kommandots ID och innehållshash före effekt. När kvittensen uteblir markeras uppdraget som osäkert. Integrationen frågar efter samma kommando och begär en färsk observation. Om journal, epok och observerat objektläge ger tillräckligt stöd kan verifieringen fortsätta utan ett nytt plock.

Om journalen säger slutfört men objektet inte kan lokaliseras är det en konflikt som kräver utredning, inte tillåtelse att välja den mest bekväma uppgiften. Om journalen gått förlorad och bilden är skymd kan korrekt beteende vara att stoppa automatiseringen och visa osäkerheten.

## 7.3 Omstarter och delvis genomförda handlingar

En omstart av integrationstjänsten får inte nollställa robotens värld. Den återläser uppdrag och utgående meddelanden från beständig lagring. Vid avbrott under sändning gör den avstämning innan nya rörelseförslag tillåts.

En omstart av Blender är svårare. Scenens senaste sparade fil kan ligga efter effektjournalen. Första versionen ska därför antingen bevara en konsistent scenögonblicksbild tillsammans med logiskt stegnummer eller uttryckligen markera att världen inte kan återställas säkert. Vid inkonsistens skapas en ny scene\_epoch och pågående uppdrag kräver manuell granskning. Att tyst läsa en gammal .blend-fil och fortsätta skulle dölja felet.

Ett uppdrag kan också avbrytas när objektet sitter i griparen. Verifieraren måste därför skilja mellan rätt destination, kvar i källan, fäst i verktyget och oklart läge. Återhämtning är inte alltid samma sak som att börja från början.

---

## 8. Observation, modell och verifiering

### 8.1 Fyra representationer av världen

Simulatorn äger det verkliga tillståndet inom modellen,  $x$ . Sensoradaptorn producerar observationen  $y$ . Brain-komponenten kan hålla en intern uppskattning eller ett minne,  $b$ . ERP-uppdraget definierar det önskade tillståndet,  $g$ . Dessa får inte blandas ihop.

En förenklad egen modell är:

```
y_t = h(x_t, calibration, occlusion, noise)
```

```
proposal_t = planner(y_t, b_t, g)
```

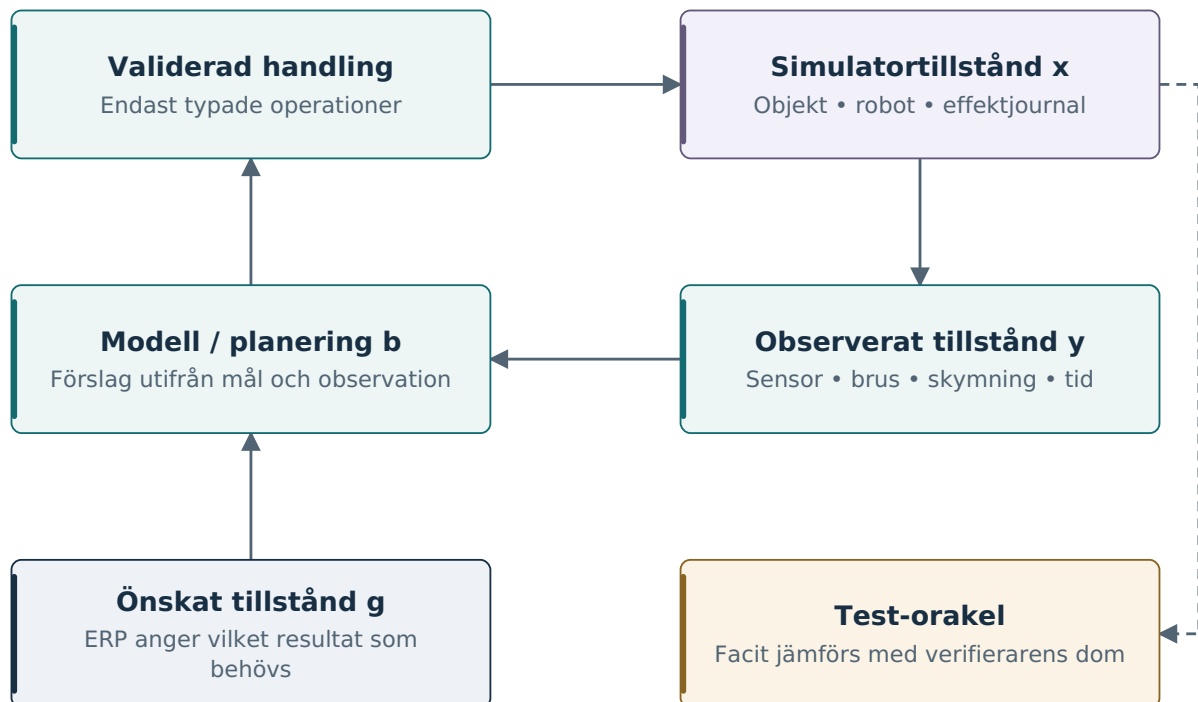
```
x_(t+1) = simulator(x_t, validated_command_t)
```

```
verdict = verifier(y_before, y_after, command_journal)
```

Uttrycken beskriver ansvar, inte en reproduktion av SICS lärandealgorithm. En klassisk databas med objektpositioner är inte automatiskt en inlärd världsmodell.

## Verklighet, observation, mål och förslag

Simulatorns facit är inte driftlogikens kunskap



Streckad pil: facit används av testet, inte som obegränsad tillgång för modellen.

Figur 5. Egen informationsmodell. Handling ändrar världen; nästa observation återkopplas till planeringen. Driftvägen får inte obegränsat facit. Test-oraklet jämför verifierarens slutsats med simulatortillståndet. Separationen undanröjer inte gemensamma modellfel.

## 8.2 Test-orakel respektive ordinarie verifierare

Test-oraklet får läsa fullständig simulatorinformation för att avgöra om systemets bedömning var rätt. Den ordinarie verifieraren får bara de sensorliknande uppgifter som scenarioinställningen medger. Annars blir testet cirkulärt: samma funktion flyttar objektet och intygar sedan att den själv gjorde rätt.

Även dessa två vägar delar simulatorns antaganden. Separationen minskar en viss typ av logiskt fel, men gör inte verifieringen oberoende av simuleringsmodellen. Korrekta metadata är inte ett bevis för korrekt synsystem i en verklig lagercell.

## 8.3 Koordinater och tid

Alla positioner uttrycks i meter, vinklar i radianer och orienteringar i en angiven konvention. En enkel koordinattransformation skrivs  $p_{\text{cell}} = R \times p_{\text{camera}} + t$ . Räkneexempel: utan rotation ger kamerakoordinaten (0,10; 0,20; 0,30) och förskjutningen (0,40; 0; 0,80) cellkoordinaten (0,50; 0,20; 1,10) meter. Detta är ett pedagogiskt exempel, inte robotkalibrering.

Varje observation har fångstid, mottagningstid och scensteg. Väggklocka används för mänskligt läsbara loggar; monotona tidsmått används för lokala tidsgränser. Simuleringstid hålls separat från verklig elapsed time. Ett bildresultat från en tidigare scene\_epoch avvisas även om objektets namn råkar vara oförändrat.

## 8.4 Deterministisk baslinje och valfri modell

Baslinjen gör ett reproducerbart val bland tillåtna objekt och rörelsesegment. Den tränar ingen modell. En valfri LLM/VLM-adapter kan ge ett strukturerat förslag, men schemakorrekthet garanterar inte semantisk korrekthet. OpenAI dokumenterar strukturerade utdata och särskild hantering av exempelvis avvisade svar. [S16]

Föreslagna felvägar är timeout, okänd objektidentitet, felaktig destination, otillräcklig observation eller avvisat modellsvar. Dessa ger ett synligt avbrott eller en dokumenterad alternativ policy. Ett självrapporterat confidence-värde ska inte behandlas som kalibrerad sannolikhet för ett lyckat grepp.

# 9. Blender, MCP och operationella gränser

## 9.1 Scen och rörelse

Scenen innehåller en generisk arm, gripare, två lådor, tre objekt, enkel transportyta, kamera och synlig cellgräns. Alla objekt har stabila identiteter. Gripning modelleras först med en kontrollerad fästning och lossning när geometriska förvillkor är uppfyllda. Ingen realistisk friktion, deformation eller sugkoppskraft antas.

Rörelsen följer definierade delsteg: närma sig, kontrollera greppvillkor, fästa objekt, lyfta, förflytta, lossa och observera igen. Systemet ändrar faktiskt scenens objektläge; det är inte en förinspelad film. En framtida IK-lösare eller fysikmotor får införas som en separat, testad utökning.

## 9.2 Blender-runtime och trådar

Blenders dokumentation varnar för osäker användning av Pythontrådar. Scenändringar ska inte göras godtyckligt från en långlivad bakgrundstråd. Timers ger ett API för schemalagda anrop. [S13][S14]

Vårt förslag är en separat bryggprocess för nätverks-I/O och en begränsad, icke-blockerande mottagning i Blender. Validerade kommandon behandlas på huvudtråden vid definierade simulatorsteg. Varje steg har en tidsbudget så att UI och händelsehantering inte blockeras av en lång operation. Vald Blender-version, tilläggsversion och scene\_hash registreras i körningens manifest.

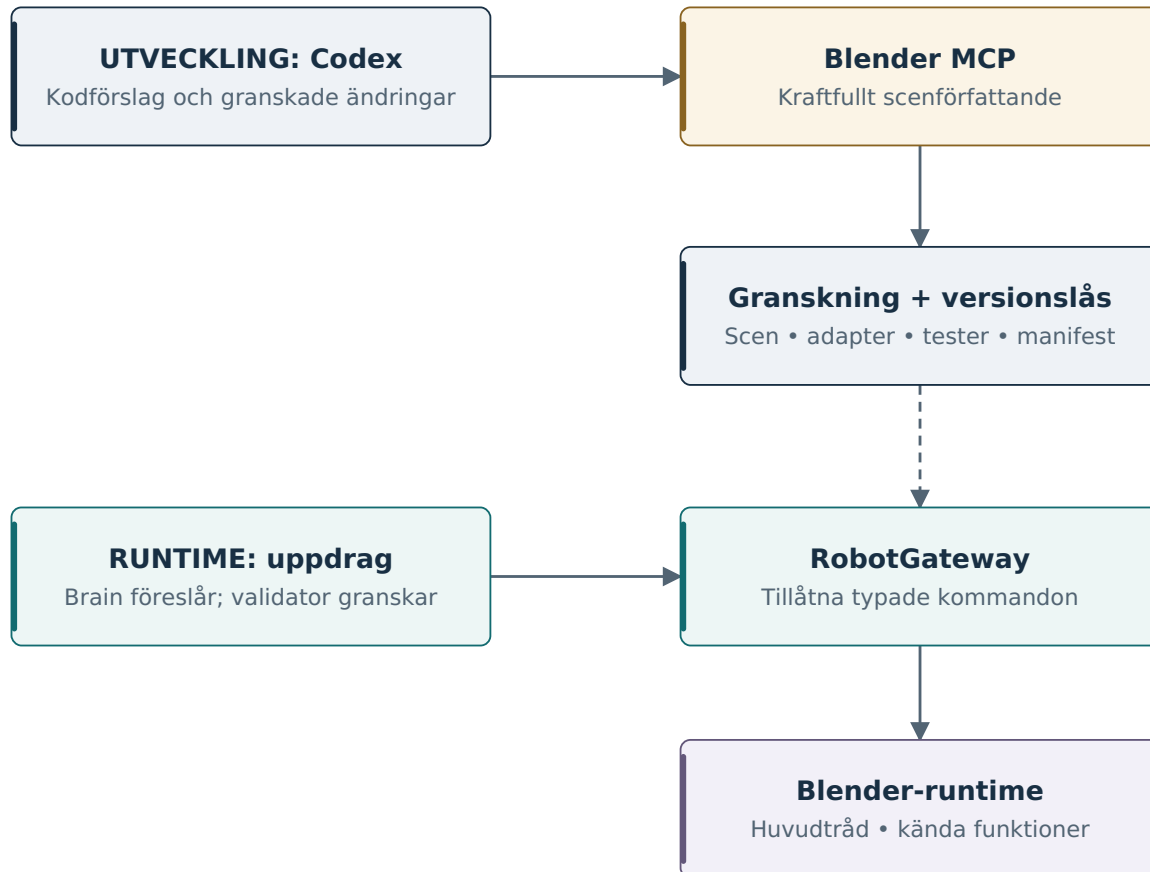
## 9.3 MCP hör till utvecklingsmiljön

Blender Lab beskriver en MCP-server med ett separat tillägg och varnar för att LLM-genererad kod kan köras utan dataskyddsräcken. Den kontrollerade sidan anger Blender 5.1 eller senare för just denna integration. [S12]

Codex kan ansluta till MCP-serverar. I projektförslaget används det för kodarbete, scenförfattande och granskning, inte som en säkerhetsklassad regulator. [S15]

### Skilj scenförfattande från robotkörning

Två behörighetsnivåer • inga fria kodkommandon i runtime



Utvecklingsverktygens behörighet är inte tillgänglig för körningens modellförslag.

Figur 6. Egen behörighetsmodell. Codex och MCP kan hjälpa till med scenförfattande i en isolerad utvecklingsmiljö. Granskade och versionshanterade artefakter förs sedan till körmiljön. RobotGateway accepterar enbart det avgränsade runtime-kontraktet.

Under utveckling kan en agent föreslå Blenderkod i en isolerad miljö. Under körning accepterar robotadaptorn endast det definierade kommandoschemat. Runtime erbjuder ingen exec, ingen godtycklig filåtkomst och inga modellvalda externa adresser. API-nycklar ligger utanför repot och aldrig i den publicerade webbsidan.

## 9.4 PLC och stopp

CellController simulerar exempelvis `ready`, `source_present`, `destination_present`, `robot_ready`, `faulted` och `stop_requested`. Före start måste definierade förvillkor vara uppfyllda. Signalerna behöver aktualitetskontroll; ett gammalt READY räcker inte.

En eventuell OPC UA-utökning ska implementera ett riktigt protokollgränssnitt med dokumenterad informationsmodell och testad klient/server-kommunikation. OPC Foundation beskriver dessa som separata delar av OPC UA. En REST-route som heter `/plc` är inte samma sak. [S18]

Stoppknappen i demonstrationen är **en simulerad operationell spärr**, inte ett nödstopp med verifierad säkerhetsfunktion. Projektet ska inte anslutas till fysisk robotutrustning med denna logik som skyddssystem.

## 10. Utvärderingsplan: hur vi avser att testa designen

### 10.1 Hypoteser och jämförelse

**H1:** Under det definierade kvittensbortfallsfallet ska journal plus avstämning undvika ett extra plockförsök som saknar stöd i observerat tillstånd.

**H2:** En verifierare med begränsad observation ska ibland lämna utfallet oavgjort, i stället för att felaktigt markera uppdraget slutfört.

**H3:** En tjänsteomstart ska inte orsaka att en redan verifierad affärseffekt utförs igen när endast ERP-kvittensen återstår.

Hypoteserna är prövbara inom simulatorns felmodell. De är inte testresultat. En medvetet naiv återförsöksstrategi kan användas som jämförelse i en isolerad testvariant. Båda varianterna ska då möta samma scener och felinjektioner. Baslinjen får inte köras mot fysisk utrustning.

### 10.2 Scenarier och godkännandekriterier

| ID  | Injicerat eller kontrollerat fall   | Förväntad egenskap                                         |
|-----|-------------------------------------|------------------------------------------------------------|
| T01 | Normalt plock                       | Ett objekt i rätt destination; korrekt ERP-kvittens.       |
| T02 | Samma order skickas två gånger      | Samma uppdrag återfås; ingen extra effekt.                 |
| T03 | Samma nyckel, ändrat innehåll       | Konflikt avvisas före rörelse.                             |
| T04 | Kvittens tappas efter effekt        | Okänt utfall synliggörs; avstämning utan blint nytt plock. |
| T05 | Kommando avvisas före start         | Ingen effekt; tydlig orsak och kontrollerat nytt försök.   |
| T06 | Integrationstjänsten startas om     | Beständig historik återläses och pågående avsikt stäms av. |
| T07 | ERP otillgängligt efter verifiering | Kundsynk återförsöks, inte robotpocket.                    |

| ID  | Injicerat eller kontrollerat fall         | Förväntad egenskap                                               |
|-----|-------------------------------------------|------------------------------------------------------------------|
| T08 | Objektet skymt efter rörelse              | Ingen falsk slutsats från att objektet inte syns.                |
| T09 | För gammal observation eller fel epok     | Planen avvisas.                                                  |
| T10 | Stopp under rörelse                       | Ingen automatisk fortsättning efter reset.                       |
| T11 | Felaktigt modellförslag                   | Validatorn hindrar exekvering och loggar orsaken.                |
| T12 | Delvis plock, objekt i griparen           | Delutfallet registreras; återstart sker inte från fel antagande. |
| T13 | Motstridiga journal- och sensoruppgifter  | Ärendet eskaleras som konflikt.                                  |
| T14 | Andra exekveraren försöker ta samma cell  | Ensam exekveringsrätt upprätthålls eller starten avvisas.        |
| T15 | Blender omstart med inkonsistent snapshot | Ny epok och granskning; inget tyst återspel.                     |
| T16 | Observation med fel koordinatenhet        | Kontraktsfel upptäcks före exekvering.                           |

Första testomgången föreslås använda tio namngivna scenfrön per scenario, med dokumenterade objektplaceringar och felpunkter. Antalet är en praktisk startpunkt, inte en statistisk styrkeberäkning. Deterministiska fall ska också testas vid flera gränser i tillståndsovergången, inte bara med flera slumpfrön.

### 10.3 Mått och invarianta villkor

**Duplicerad effekt:** antal uppdrag där fler effekter än beställt har inträffat, dividerat med antal uppdrag.

**Falskt slutförande:** antal uppdrag märkta slutförda trots att test-oraklet visar fel slutläge, dividerat med antalet slutförda uppdrag. Om nämnaren är noll redovisas måttet som ej tillämpligt.

**Oavgjorda utfall:** antal uppdrag där verifieraren saknar tillräckligt underlag. Ett lägre värde är inte automatiskt bättre om det köps genom fler felaktiga godkännanden.

**Återhämtningstid:** tid från upptäckt fel till verifierad lösning eller eskalering. Rapportera både simulerade steg och verklig elapsed time, med maskin- och versionsuppgifter.

Invarianta villkor är bland annat att gamla scene\_epoch-kommandon inte körs, att endast en aktör ändrar roboten åt gången och att färdig kundorder bygger på en lagrad verifieringspost. Noll observerade fel i ett litet prov ska inte tolkas som noll verklig felrisk.

## 10.4 Reproduktionspaket

Varje körning ska kunna exportera Git-commit, konfiguration, scene\_hash, seed, schema- och modellversion, inskickade kommandon, händelseföljd, observationer, orakelutfall och mätvärden. Modellkörningar redovisas separat från den deterministiska baslinjen.

Nätverkskostnader och externa anrop ska vara synliga och begränsade.

Ett misslyckat experiment sparas lika noggrant som ett lyckat. Presentationen får inte välja endast vackra videosekvenser och dölja resten av testpopulationen.

# 11. Genomförande, dokumentation och underhåll

## 11.1 Byggordning

**Ettapp A: kontrakt och testbar kärna.** Skapa modeller, beständig uppdragslogg, kundmock och en robotstub utan grafik. Godkänn T01–T07 innan scenarbete blir huvudfokus.

**Ettapp B: Blender som värld.** Inför scenmetadata, begränsad runtime, observation och verifiering. Demonstrera ett normalt plock och ett kvittensbortfall med samma orderflöde.

**Ettapp C: svåra fel.** Lägg till stopp, stale observation, delutfall, epoker och restart-konflikter. Dokumentera vilka fall som ännu inte kan lösas automatiskt.

**Ettapp D: tillägg.** Lägg till valfri bildmodell eller ett verkligt lokalt OPC UA-gränssnitt först när baslinjen är stabil. Fotorealism, komplex fysik och flera robotar prioriteras sist.

## 11.2 Föreslagen struktur för senare simulatorimplementation

```
src/
  customer/      # ERP/WMS-mock och kundkvittens
  integration/   # uppdrag, outbox, återhämtning
  contracts/     # typade API- och händelsemodeller
  brain/         # deterministisk och valfri modelladapter
  cell/          # operationella signaler och ägarskap
  robot/         # gateway, kapabiliteter och journal
  verification/  # observerad effekt och avstämning
blender/
  scene_setup.py
  runtime_adapter.py
tests/
  unit/
  contract/
  integration/
  scenarios/
runs/            # lokala kördata; inte hemligheter i Git
reports/        # detta rapportpaket
```

Detta är en målstruktur, inte en lista över redan implementerade simulatorfiler. Dokumentationsbygget hålls separat från runtime.

### 11.3 Definition av färdig demonstration

Demonstrationen är färdig först när en annan person kan följa startinstruktionen, skapa en order, se en verklig scenändring, följa dess ID genom tidslinjen och återköra kvittensbortfallsfallet. Ett statusmanifest ska ange genomförda tester och kända begränsningar. Skärminspelning är en reservväg, inte en ersättning för testbarhet.

## 12. Diskussion, begränsningar och slutsats

Det mest överförbara i projektet är gränssnitten och resonemanget om osäkerhet: affärsavsikt, exekveringsförsök och effekt behöver olika identiteter och bevis. Den minst överförbara delen är den förenklade världen. Fästning av ett objekt i en gripare kan inte belägga robust gripning, och en kontroll i SQLite kan inte belägga säker robotrörelse.

Det finns risk för gemensamma modellfel mellan simulator och verifierare. Scenarierna är få, produkter förenklade och hårdvarans dynamik saknas. En framtida verklig integration kräver nya adapterspecifikationer, riskbedömning, fysisk kalibrering, driftsättningsprov och verksamhetsförankring. Rapport 2 utvecklar dessa gränser.

**Slutsatsen är att projektet är motiverat som ett avgränsat integrations- och återhämtningstest, inte som en kopia av SICS AI:s robotbrain.** En liten demo med ärlig osäkerhet, spårbara beslut och reproducerbara fel är vetenskapligt mer användbar än en större animation som ger sken av verifierad fysisk förmåga.

Den fortsatta diskussionen bör pröva designens antaganden: var den verkliga plattformens gränser går, vilka fel som är vanligast och vilken evidens som krävs för att ett plock ska räknas som lyckat. Inga experimentella resultat för RobotOps Twin redovisas i version 1.0.

## Referenser och källbegränsningar

Referenserna avser avgränsade sakuppgifter. Diagram, kontrakt och testplan är egna förslag. Gemensamt kontrolldatum: 1 oktober 2026.

### [S01] Systemingenjör — annons 4464336388

Arbetsgivarens rollbeskrivning

Superintelligence Computing Systems. Utan säkert publiceringsdatum. Kontrollerad 2026-10-01.

<https://se.linkedin.com/jobs/view/systemingenj%C3%B6r-at-superintelligence-computing-systems-4464336388>

Offentligt sökindexerad annons kontrollerad. Används som kravunderlag; inte som bevis för intern teknikstack.

Direktsidan kunde inte återläsas konsekvent.

### [S02] Systemingenjör — alternativ annonsversion 4465300388

Arbetsgivarens rollbeskrivning

Superintelligence Computing Systems. Utan säkert publiceringsdatum. Kontrollerad 2026-10-01.

<https://se.linkedin.com/jobs/view/systemingenj%C3%B6r-at-superintelligence-computing-systems-4465300388>

Offentligt sökindexerad annons med delvis annan formulering av erfarenhetskrav. Versionerna ska inte behandlas som ett enda exakt avtal.

---

### [S03] Foundation AGI model for Industrial Robots — HYPER: reporting

Självrapporterat projektarbete

SICS AI, deltagarrapport hos Europeiska kommissionen/CORDIS. Uppdaterad 1 juni 2026. Kontrollerad 2026-10-01.

<https://cordis.europa.eu/project/id/101218436/reporting>

Projekt-ID 101218436. Beskrivningen av modell och robotkontroll är deltagarens redovisning, inte oberoende reproduktion. DOI för projektet: 10.3030/101218436.

---

### [S04] Simulation In The Loop: Real-Time Robot Action Success Verification with a Digital Twin

Examensarbete — bibliografisk uppgift

Axel Kaliff, KTH. 2025. Kontrollerad 2026-10-01.

<https://kth.diva-portal.org/smash/record.jsf?pid=diva2%3A1988006>

Titel och författarsamband stöds av examinatorns offentligt indexerade KTH-sida. DiVA-fulltexten kunde inte hämtas vid denna kontroll. Inga detaljerade metod- eller resultatpåståenden återges.

Kompletterande primärkälla: <https://www.csc.kth.se/~fpokorny/>

---

### [S05] Project Autograsper: Custom Grasper Development Guide

Öppen primärdokumentation

CloudGripper-projektet. Löpande repositorydokumentation. Kontrollerad 2026-10-01.

<https://github.com/cloudgripper/autograsper/blob/main/README.md>

README granskad via GitHub. Git-blob: 3cb1da256bc5147ef52a56327da476f110c4e6c2. Forskningsverktyg; inte SICS AI:s produktkod. Ingen kod eller fjärrrobotkörning har återanvänts i rapportpaketet.

---

### [S06] R900: Understanding the Cost-Effectiveness of Random Exploration from 900 Hours of Robotic Data Collection

Forskningspreprint

Shutong Jin, Axel Kaliff, Ruiyu Wang, Muhammad Zahid och Florian T. Pokorny. arXiv v2, 17 september 2025. Kontrollerad 2026-10-01.

<https://arxiv.org/abs/2503.23571>

DOI: 10.48550/arXiv.2503.23571. Används för datainsamling, episodhantering och återställning som designinspiration. Projektet reproducerar inte experimenten eller resultaten.

---

### [S07] Nowaste Logistics Advances Automation with Cognibotics Robot Picking Cell

Leverantörens installationsbeskrivning

Cognibotics. 5 januari 2026. Kontrollerad 2026-10-01.

<https://www.cognibotics.com/en/articles/nowaste-logistics-advances-automation-with-cognibotics-robot-picking-cell>

Beskriver andra cellen, AutoStore, SICS AI-vision och Juliet & Romeo. Leverantörsuppgifter, inte publicerad intern API-specifikation eller oberoende driftsrevision.

---

### [S08] HKM1800 — robot for intralogistics and warehouse automation

Tillverkardokumentation

Cognibotics. Utan datum. Kontrollerad 2026-10-01.

<https://www.cognibotics.com/en/products/hkm>

Används för att skilja verklig hybridkinematik från projektets generiska arm. Tillverkarens kapacitetsuppgifter är inte simulatorresultat.

---

---

## [S10] SICS AI: HPC Access to LUMI Supercomputer for AI and Robotics

Teknisk samarbetsbeskrivning

ENCCS. 12 september 2025. Kontrollerad 2026-10-01.

<https://enccs.se/2025/09/news/sics-hpc-access-lumi-supercomputer-ai-robotics-3/>

Dokumenterar arbete med beräkningsmiljö och GPU-träning. Fastställer inte kundernas aktuella inferensarkitektur.

---

## [S11] SICS AI investerar i NVIDIA DGX B200 från Aixia

Leverantörens pressmeddelande

Aixia Group AB. 17 juni 2025. Kontrollerad 2026-10-01.

<https://news.cision.com/se/aixia-group-ab--publ-/r/sics-ai-investerar-i-nvidia-dgx-b200-fran-aixia---accelererar-utvecklingen-av-ai-hjarna-for-nasta-ge%2Cc4165473>

Bekräftar offentliggjord DGX B200-affär med drifttjänster. Inte ett test av modeller eller belägg för var dagens maskin är placerad.

---

## [S12] MCP Server

Officiell verktygsdokumentation

Blender Lab / Blender Foundation. Löpande dokumentation. Kontrollerad 2026-10-01.

<https://www.blender.org/lab/mcp-server/>

Dokumenterar separat MCP-integration och varnar för exekvering av modellgenererad kod. Sidan anger Blender 5.1 eller senare för integrationen. Faktisk projektkompatibilitet återstår att testa.

---

## [S13] Blender Python API: Python Threads are Not Supported

Officiell API-dokumentation

Blender Foundation. Dokumentationsversion 5.0. Kontrollerad 2026-10-01.

[https://docs.blender.org/api/5.0/info\\_gotchas\\_threading.html](https://docs.blender.org/api/5.0/info_gotchas_threading.html)

Används för tråd- och processgränsen i den föreslagna runtime-designen. Vald körversion ska låsas och testas separat.

---

## [S14] Blender Python API: Application Timers (bpy.app.timers)

Officiell API-dokumentation

Blender Foundation. Dokumentationsversion 3.6. Kontrollerad 2026-10-01.

<https://docs.blender.org/api/3.6/bpy.app.timers.html>

Beskriver callback-baserad schemaläggning och huvudtrådsexekvering. Dokumentationsreferensen är inte ett krav att köra Blender 3.6.

---

## [S15] Codex: Model Context Protocol

Officiell produktdokumentation

OpenAI. Löpande dokumentation. Kontrollerad 2026-10-01.

<https://developers.openai.com/codex/mcp>

Beskriver Codex anslutning till MCP-servrar. Motiverar utvecklingsverktyg, inte användning som säkerhetsklassad robotcontroller.

---

---

## [S16] Structured outputs

Officiell API-dokumentation

OpenAI. Löpande API-dokumentation. Kontrollerad 2026-10-01.

<https://developers.openai.com/api/docs/guides/structured-outputs>

Schemaformat, avvisade svar och begränsningar. Ett giltigt schema är inte bevis för ett korrekt eller säkert handlingsförslag.

---

## [S17] Making retries safe with idempotent APIs

Primär teknisk designbeskrivning

Amazon Web Services, Builders' Library. Utan datum. Kontrollerad 2026-10-01.

<https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>

Stabila begärandeidentiteter och idempotent semantik. Projektets slutsatser om fysiska effekter är egen designanalys, inte en utlovad AWS-garanti för robotar.

---

## [S18] OPC Unified Architecture

Protokolläggarens dokumentation

OPC Foundation. Löpande dokumentation. Kontrollerad 2026-10-01.

<https://opcfoundation.org/about/opc-technologies/opc-ua/>

Informationsmodell och kommunikationstjänster. En lokal signalmodell eller REST-route innebär inte implementerat OPC UA-stöd.

---

## [S19] Creating a GitHub Pages site; configuring a publishing source

Officiell publiceringsdokumentation

GitHub Docs. Löpande dokumentation. Kontrollerad 2026-10-01.

<https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site>

Pages används för statiska rapporter och nedladdningar, inte för Python eller Blender-runtime.

Kompletterande primärkälla: <https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site>

---

# Simulatorn och den verkliga robotcellen

Avgränsning, realism och överförbarhet

Rami Halabi

Version 1.0 · 1 oktober 2026

Oberoende tekniskt rapportpaket · Svenska

**Status: design och forskningsunderlag.** Simulatorn beskrivs som ett planerat system. Inga empiriska robotresultat eller säkerhetsgarantier hävdas.

AI-assisterad text- och publiceringsframställning.

Ej sakkunniggranskad. Inte framtagen av eller godkänd av SICS AI.

Rapporternas källor, diagram och byggsript finns i projektets repository.

# Innehåll

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| Sammanfattning.....                                                            | 3  |
| 1. Jämförelsens tre nivåer .....                                               | 3  |
| 2. Vad vi vet, vad vi förenklar och vad vi inte vet.....                       | 4  |
| 3. Vilken typ av realism är viktig?.....                                       | 6  |
| 4. "Digital twin" är ett projektnamn, inte en valideringsstämpel .....         | 7  |
| 5. Den viktigaste gränsen: kontroll av kommando kontra kontroll av effekt..... | 7  |
| 6. Vad ett byte till verklig hårdvara skulle kräva.....                        | 8  |
| 7. Påståenden som är rimliga respektive missvisande.....                       | 8  |
| 8. Vad som avsiktligt ligger utanför projektet.....                            | 9  |
| 9. Risker, motåtgärder och kvarvarande osäkerhet.....                          | 9  |
| 10. Slutsats och kontrollista för presentation .....                           | 10 |
| Referenser och källbegränsningar .....                                         | 10 |

---

## Sammanfattning

Den här rapporten skiljer **RobotOps Twins planerade simulator** från dels en generell industriell robotcell, dels den begränsade bild av SICS AI:s miljö som finns i offentliga källor. Dessa två jämförelseobjekt är inte samma sak. Offentlig information räcker inte för att beskriva företagets fullständiga interna system.

Simulatorn ska efterlikna informationsflödet från kundorder till verifierat plock och återkoppling. Den ska bevara vissa integrationsproblem: dubletter, osäkra utfall, återhämtning, separata systemägare och motstridiga observationer. Den ska däremot förenkla robotmekanik, sensorsystem, säkerhet, produktvariation, nätverk och driftorganisation.

**Det som senare kan demonstreras är programvarans beteende under definierade simuleringsantaganden.** Det är inte ett bevis för fysisk säkerhet, kommersiell driftsäkerhet, ett fungerande SICS-gränssnitt eller generell intelligens. Projektet befinner sig i denna version på designstadiet.

---

## 1. Jämförelsens tre nivåer

Vi använder tre nivåer för att undvika att generella branschbegrepp tillskrivs ett enskilt bolag.

**Nivå A: dokumenterad offentlig uppgift.** Exempelvis anger SICS rollannons kundintegration, medan Cognibotics beskriver en konkret Nowaste-cell. Uppgifterna citeras och attribueras. [\[S01\]](#) [\[S07\]](#)

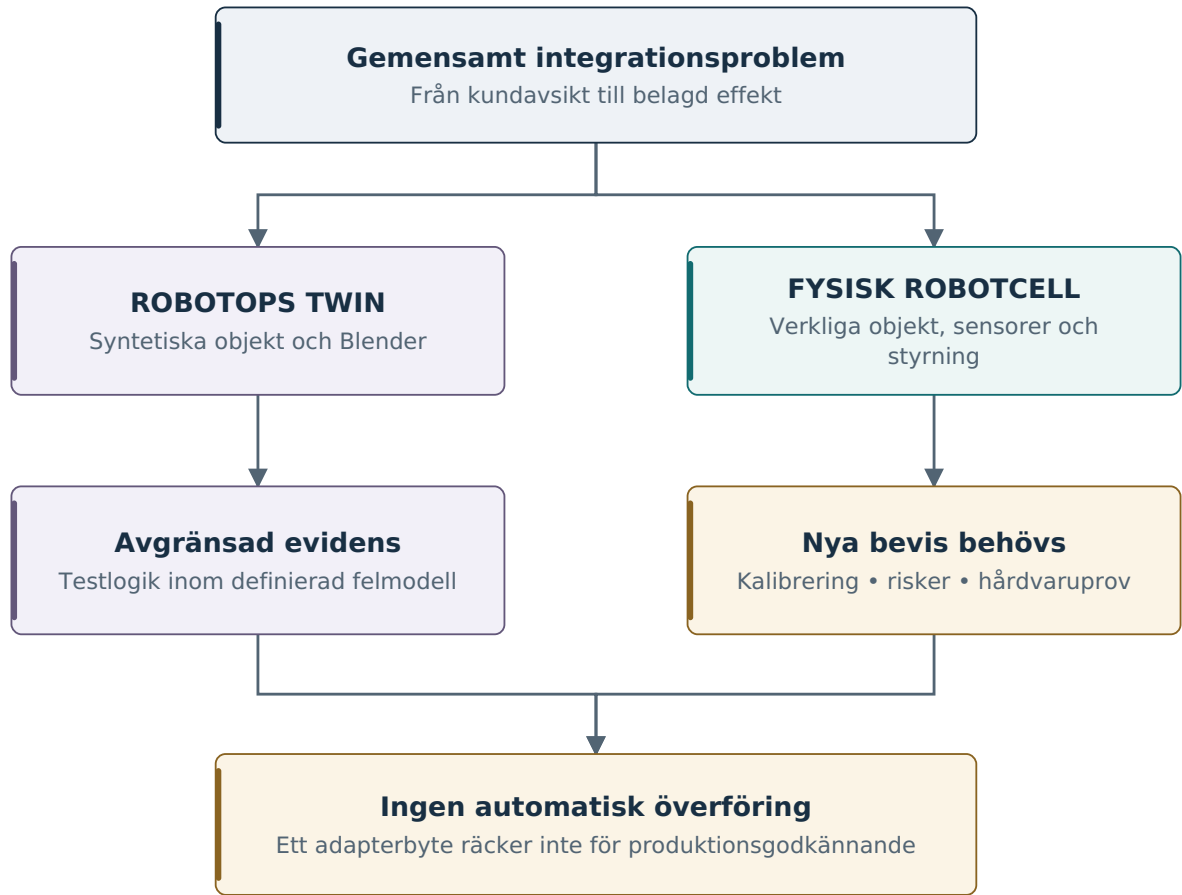
**Nivå B: generell integrationsfråga.** Exempelvis måste ett system skilja mellan förlorad kommunikation och en känd utebliven sidoeffekt. Det är en fråga om distribuerad systemdesign, inte ett fynd om hur SICS har implementerat sin lösning. [\[S17\]](#)

**Nivå C: vårt föreslagna val.** SQLite, en Blender-arm, en deterministisk Brain-adapter och ett visst tillståndsdigram är egna beslut. De ska kunna ändras utan att källorna ändras.

Jämförelsen är alltså inte en omvänd konstruktion av SICS AI:s produkt. Den identifierar vilka aspekter en liten demonstrator kan representera och var motsvarigheten upphör.

Gemensam fråga — olika beviskrav

Simulatorn är inte en validerad kopia av en fysisk cell



Figur 7. Egen jämförelse. Kontrakt, identiteter, felhantering och observerbarhet kan studeras i båda miljöerna. Fysik, sensorer, säkerhet och driftsvillkor kräver ny validering vid övergång till verklig utrustning. Figuren beskriver ingen intern SICS-arkitektur.

2. Vad vi vet, vad vi förenklar och vad vi inte vet

2.1 Affärssystem och integration

| Aspekt       | Offentlig eller generell verklighetsbild                            | RobotOps Twins mål                                       |
|--------------|---------------------------------------------------------------------|----------------------------------------------------------|
| Kundflöde    | Rollen omfattar kundens API/WMS/ ERP. [S01]                         | Syntetiska order, tre SKU:er och ett plock per orderrad. |
| Affärsregler | Exakta kundregler är inte offentliga.                               | En enkel regel: rätt exemplar till rätt orderlåda.       |
| Integration  | Leverantörsmaterial beskriver lagerstyrning runt robotcellen. [S07] | Separat kundadapter och kvittenskontrakt.                |

| Aspekt        | Offentlig eller generell verklighetsbild                            | RobotOps Twins mål                                           |
|---------------|---------------------------------------------------------------------|--------------------------------------------------------------|
| Inventering   | Verklig redovisning, reservation och korrigering är inte kartlagda. | Förenklad reservations- och saldomodell utan bokföring.      |
| Kommunikation | Det faktiska protokollet och garantierna är okända.                 | Lokala API:er med injicerad fördröjning och kvittensförlust. |

En mock kan visa hur kontrakt hanteras. Den kan inte visa att alla verkliga affärsregler är förstådda. Returer, substitution, batcher, förpackningshierarkier, lagersaldokonflikter och flera samtidiga kunder ligger utanför den första versionen.

Ett lyckat plock kan också vara olika saker i olika affärsprocesser: flyttat objekt, scannad artikel, klar orderrad eller skickklar försändelse. Simulatoren ska därför uttryckligen definiera sin sluttillståndsregel. Den bör inte använda ordet "levererad" för en artikel som bara har flyttats mellan två lådor.

## 2.2 AI, robot och rörelse

| Aspekt         | Offentlig uppgift eller kunskapslucka                                                         | Simulatorns avgränsning                                          |
|----------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Robotbrain     | HYPER beskriver en proprietär modell. <a href="#">[S03]</a>                                   | Ett gränssnitt, inte en återimplementation.                      |
| Modellstyrning | Offentliga texter beskriver olika kontrollnivåer. <a href="#">[S03]</a> <a href="#">[S07]</a> | Uppgiftsnivå i MVP; lednivå endast ett framtida adapterkontrakt. |
| Robotmekanik   | HKM1800 har hybridkinematik enligt tillverkaren. <a href="#">[S08]</a>                        | Generisk arm; inga HKM-prestandapåståenden.                      |
| Gripning       | Verkliga objekt och verktyg påverkar resultatet.                                              | Regelstyrd fästning/lossning av syntetiska objekt.               |
| Perception     | Den exakta sensorkedjan är inte offentlig.                                                    | Syntetisk observation, med valfri rendering och bildmodell.      |
| Inlärnin       | Algoritm, data och vikter är inte tillgängliga för denna studie.                              | Ingen träning i baslinjen; versionshanterad planeringslogik.     |

En animation kan vara begriplig utan att dess moment, kontaktkrafter eller ledhastigheter motsvarar en verklig robot. Det är tillåtet i en integrationsdemo om det är tydligt deklarerat. Problemet uppstår när visuellt övertygande rörelse används som underlag för påståenden om dynamisk noggrannhet.

## 2.3 PLC, beräkning och drift

| Aspekt | Verklighetsanknytning                                     | Vad demonstratorn inte styrker                          |
|--------|-----------------------------------------------------------|---------------------------------------------------------|
| PLC    | Finns i rollens integrationsområde. <a href="#">[S01]</a> | Att ett logiskt signalobjekt är ett testat PLC-program. |

| Aspekt   | Verklighetsanknytning                                               | Vad demonstratorn inte styrker                                         |
|----------|---------------------------------------------------------------------|------------------------------------------------------------------------|
| OPC UA   | Har informationsmodell och kommunikationstjänster. <b>[S18]</b>     | Att ett JSON-API innebär OPC UA-stöd.                                  |
| GPU      | LUMI-arbete och DGX-leverans är offentliggjorda. <b>[S10] [S11]</b> | Att simulatormen motsvarar träning på B200.                            |
| Drift    | Rollen omfattar system i kundmiljö. <b>[S01]</b>                    | Tillgänglighet över månader, jourförmåga eller verkligt serviceansvar. |
| Säkerhet | Verklig utrustning har egna säkerhetskrav och risker.               | Funktionell säkerhet, validerat nödstopp eller CE-underlag.            |

### 3. Vilken typ av realism är viktig?

#### 3.1 Semantisk realism

Projektets viktigaste mål är att betydelsen i meddelandena ska vara konsekvent. Kundens begäran, integrationens avsikt och robotens exekveringsförsök ska inte ha samma otydliga "klart"-flagga. Begrepp som accepterat, utfört, verifierat och kundkvitterat måste kunna skiljas åt.

Detta går att demonstrera med enkla objekt och låg grafisk detaljnivå. En grå låda som hamnar i fel destination är ett bättre testfall än en fotorealistic kartong vars position aldrig kontrolleras.

#### 3.2 Beteendemässig realism

Systemet ska kunna visa plausibla felreaktioner: en tjänst förlorar svaret, ett objekt är skymt, en kvittens återlevereras och en gammal plan når en återställd scen. Reaktionerna måste följa definierade regler, inte en demonstration som är skriptad för att alltid lyckas.

Vi modellerar dock inte hela felpopulationen i en riktig anläggning. Inga slutsatser om sannolikheten för driftstopp kan dras från att ett antal utvalda fel har hanterats.

#### 3.3 Fysikalisk och perceptuell realism

Fysikalisk realism är låg i första versionen. Simulatormen kan kontrollera geometriska villkor men saknar exempelvis verklig kontaktfysik, deformation och materialvariation. Perceptuell realism är också begränsad om objektens identiteter kommer från metadata.

En valfri bildmodell höjer inte automatiskt realismen till produktionsnivå. Bilder från Blender kan avvika systematiskt från verkliga kamerabilder. Dessutom kan modellens klassificering vara korrekt samtidigt som koordinater eller tidpunkter är fel.

### 3.4 Operationell realism

Loggning, idempotens och återhämtning kan vara ganska realistiska trots en enkel värld. Däremot saknas människor som flyttar gods, underhållspersonal, skiftbyten, leverantörsgränser, nätverkssegment och verkliga incidentprocesser. Dessa ska inte döljas bakom ett dashboard-uttryck som "production ready".

---

## 4. "Digital twin" är ett projektnamn, inte en valideringsstämpel

RobotOps Twin ska i första hand beskrivas som en **simulerad cell med digital-tvilling-inspirerad systemdesign**. Vi saknar en verklig referenscell, en mätkedja som synkroniserar den och en utvärdering av modellens avvikelse från verkligheten.

Axels examensarbetstitel ger en relevant tematisk koppling, men denna rapport har inte verifierat hela examensarbetets metod eller resultat. Det vore därför fel att säga att vår efterhandsverifiering i Blender är samma metod eller att en KTH-studie har validerat vår design. [S04]

Ett framtida digitalt tvillingarbete behöver definiera vilken tillgång som avbildas, vilka tillstånd som uppdateras, hur ofta uppdatering sker och vilka avvikelser som accepteras för den avsedda användningen. Det räcker inte att två skärmar visar samma robotform.

---

## 5. Den viktigaste gränsen: kontroll av kommando kontra kontroll av effekt

### 5.1 Vad som faktiskt kan testas

Inom vår modell kan test-oraklet räkna hur många gånger ett objekt har flyttats och om affärsstatus motsvarar slutläget. Därmed går det att upptäcka logiska fel: samma order skapar två effekter, gamla kommandon körs efter scenreset eller en kundkvittens orsakar ett nytt robotplock.

Den ordinarie verifieraren ska däremot arbeta med begränsad observation. Att test-oraklet vet exakt var objektet finns betyder inte att driftlogiken har samma kunskap.

### 5.2 Vad som inte kan bevisas

Ett lyckat test visar inte att en fysisk robot aldrig dubbelplockar. Ett plock kan misslyckas på sätt som vår modell inte innehåller. Två funktioner kan dela samma felaktiga antagande och ändå vara överens.

Exempel: simulatorn flyttar ett objekt genom att ändra dess parent och verifieraren läser samma parent-fält. Då kan båda rapportera framgång utan att vi har testat en oberoende sensorväg. Därför ska test-orakel, observerad värld och beslut särskiljas, och kvarvarande gemensamma beroenden redovisas.

### 5.3 Oavgjort är ett legitimt resultat

Vid kvittensförlust kan journal och ny observation stödja ett slutförande. Men om journalen är borta och objektet skymt bör systemet inte gissa. En tydlig `MANUAL_REVIEW` är i det fallet mer korrekt än att forcera flödet till grönt.

Demonstrationen ska visa både ett återhämtningsbart fel och ett ärligt oavgjort fall. Då blir det tydligt att reconciliation inte är en magisk funktion som alltid vet vad som hände.

## 6. Vad ett byte till verklig hårdvara skulle kräva

En adaptergräns är värdefull men ett hårdvarubyte är ett integrationsprojekt. Tabellen nedan beskriver ett föreslaget införandeförfarande, inte SICS AI:s process.

| Steg                           | Nödvändigt underlag                                             | Föreslagen beslutsgrind                                |
|--------------------------------|-----------------------------------------------------------------|--------------------------------------------------------|
| 1. Kontraktsgranskning         | Leverantörens API, felkoder, enheter och kvittenser.            | Alla kommandomappningar och okända fall dokumenterade. |
| 2. Risk- och ansvarsfördelning | Ansvariga för cell, robot, arbetsmiljö och säkerhetsfunktioner. | Ingen rörelse med oklart säkerhetsansvar.              |
| 3. Geometri och kalibrering    | Verkliga koordinatsystem, verktyg och tillåtna områden.         | Mätningar och verifiering mot avsedd uppgift.          |
| 4. Kontrollerat hårdvaruprov   | Avgränsad testmiljö och godkända rutiner.                       | Rörelse, stopp och felrespons granskade av ansvariga.  |
| 5. Affärssystemtest            | Riktiga kundregler och representativa data.                     | Order- och inventeringssemantik accepterad av kund.    |
| 6. Driftprov                   | Övervakning, återställning, versionsbyte och support.           | Mätbara acceptanskriterier och kända begränsningar.    |

Varken rapportpaketet eller simulatorns planerade logik ska användas som säkerhetsinstruktion för fysisk utrustning. Funktionell säkerhet måste hanteras av personer med rätt ansvar och kompetens i den aktuella anläggningen.

## 7. Påståenden som är rimliga respektive missvisande

| Rimlig formulering                                                | Missvisande formulering                                  |
|-------------------------------------------------------------------|----------------------------------------------------------|
| "Jag har specificerat en simulerad order-robot-kedja."            | "Jag har byggt SICS system."                             |
| "Det här är ett test av idempotens inom en definierad felmodell." | "Det här garanterar exactly-once i den fysiska världen." |
| "Blender ersätter den fysiska cellen i demonstrationen."          | "Modellen är en validerad tvilling av Nowastes cell."    |

| Rimlig formulering                               | Missvisande formulering                             |
|--------------------------------------------------|-----------------------------------------------------|
| "En deterministisk adapter föreslår handlingar." | "Jag har återskapat deras inlärd robotbrain."       |
| "Stoppet är en logisk simulering."               | "Systemet har ett certifierat nödstopp."            |
| "Bildmodellen lämnar ett förslag som valideras." | "Schemakorrekt AI-utdata är säkert att exekvera."   |
| "OPC UA är ett planerat tillägg."                | "PLC/OPC UA är klart eftersom jag har en signalvy." |
| "Det här behöver testas på verklig hårdvara."    | "Byt adapter så är allt produktionsklart."          |

När simulatorn senare är byggd ska formuleringarna uppdateras till dåtid endast för de delar som faktiskt går att visa. Testresultat ska länka till en körning och en commit, inte bara till en skärmbild.

## 8. Vad som avsiktligt ligger utanför projektet

Den första versionen omfattar inte AGI-forskning, egen foundationmodell, full HKM1800-kinematik, säkerhets-PLC, komplex kontaktfysik, storskalig GPU-träning, flerrobotsamordning, verklig AutoStore-integration eller ett komplett ERP. Den omfattar inte heller säkerhetscertifiering, juridisk bedömning eller revision av SICS AI.

Detta är inte ett argument för att dessa områden är oviktiga. Avgränsningen gör i stället att de delar som faktiskt byggs kan förstås, testas och diskuteras. En liten design med uttryckliga begränsningar är lättare att granska än en bred demo som använder många tekniska namn utan motsvarande implementation.

GitHub Pages-publiceringen är en dokumentationsleverans. Den ska inte förväxlas med en molndriftsatt simulator. Python, lokal databas, Blender och eventuella modellnycklar hör till en separat körmiljö. [\[S19\]](#)

## 9. Risker, motåtgärder och kvarvarande osäkerhet

| Risk                                     | Föreslagen motåtgärd                                    | Kvarvarande begränsning                                          |
|------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------|
| Visuell realism övertygar mer än tester. | Visa status och effektjournal tillsammans med grafiken. | Grafiken kan ändå ge ett starkare intryck än modellen motiverar. |
| Verifieraren läser facit.                | Separera sensorväg och test-orakel.                     | Båda delar simulatorns grundantaganden.                          |
| För många verktyg försämrar leveransen.  | En lokal cell och ett aktivt uppdrag.                   | Ingen skalbarhetsvalidering.                                     |
| LLM gör demonstrationen instabil.        | Deterministisk baslinje och tydlig felpolicy.           | Baslinjen visar inte modellens intelligens.                      |

| Risk                                  | Föreslagen motåtgärd                              | Kvarvarande begränsning                                    |
|---------------------------------------|---------------------------------------------------|------------------------------------------------------------|
| Återställning raderar bevis.          | Epoker, beständig historik och export av körning. | En inkonsistent fysiskliknande värld kan kräva granskning. |
| Agentverktyg får för stor behörighet. | Isolerat scenförfattande; inget MCP i runtime.    | Utvecklingsmiljön behöver fortfarande skyddas.             |

Blender Labs egen varning om exekvering av modellgenererad kod är motiv för separationen mellan författande och drift. Det är inte ett påstående om att ett visst MCP-paket har använts eller säkerhetsgranskats i denna version. [S12]

## 10. Slutsats och kontrollista för presentation

Den rimliga ambitionen är **hög tydlighet i integrationssemantik och låg, öppet deklarerad fysisk realism**. Projektet ska göra det möjligt att diskutera var beslut tas, vem som äger tillstånd och vad som händer när observation och journal inte stämmer överens.

Före varje demonstration ska projektägaren kunna besvara följande i vanliga ord: vad är implementerat, vad är simulerat, vilka källor inspirerade valet, vilka tester har körts och vad kräver fortfarande verklig utrustning? När dessa svar är tydliga blir begränsningarna en del av det tekniska resonemanget, inte något som behöver döljas.

**Rapportens slutsats är inte att simulatorn motsvarar den verkliga miljön. Slutsatsen är att den kan bevara utvalda, betydelsefulla integrationsproblem samtidigt som den gör deras gränser synliga.**

## Referenser och källbegränsningar

Referenserna avser avgränsade sakuppgifter. Diagram, kontrakt och testplan är egna förslag. Gemensamt kontrolldatum: 1 oktober 2026.

### [S01] Systemingenjör — annons 4464336388

Arbetsgivarens rollbeskrivning

Superintelligence Computing Systems. Utan säkert publiceringsdatum. Kontrollerad 2026-10-01.  
<https://se.linkedin.com/jobs/view/systemingenj%C3%B6r-at-superintelligence-computing-systems-4464336388>  
Offentligt sökindexerad annons kontrollerad. Används som kravunderlag; inte som bevis för intern teknikstack. Direktsidan kunde inte återläsas konsekvent.

### [S03] Foundation AGI model for Industrial Robots — HYPER: reporting

Självrapporterat projektarbete

SICS AI, deltagarrapport hos Europeiska kommissionen/CORDIS. Uppdaterad 1 juni 2026. Kontrollerad 2026-10-01.  
<https://cordis.europa.eu/project/id/101218436/reporting>  
Projekt-ID 101218436. Beskrivningen av modell och robotkontroll är deltagarens redovisning, inte oberoende reproduktion. DOI för projektet: 10.3030/101218436.

---

## [S04] Simulation In The Loop: Real-Time Robot Action Success Verification with a Digital Twin

Examensarbete — bibliografisk uppgift

Axel Kaliff, KTH. 2025. Kontrollerad 2026-10-01.

<https://kth.diva-portal.org/smash/record.jsf?pid=diva2%3A1988006>

Titel och författarsamband stöds av examinatorns offentligt indexerade KTH-sida. DiVA-fulltexten kunde inte hämtas vid denna kontroll. Inga detaljerade metod- eller resultatpåståenden återges.

Kompletterande primärkälla: <https://www.csc.kth.se/~fpokorny/>

---

## [S07] Nowaste Logistics Advances Automation with Cognibotics Robot Picking Cell

Leverantörens installationsbeskrivning

Cognibotics. 5 januari 2026. Kontrollerad 2026-10-01.

<https://www.cognibotics.com/en/articles/nowaste-logistics-advances-automation-with-cognibotics-robot-picking-cell>

Beskriver andra cellen, AutoStore, SICS AI-vision och Juliet & Romeo. Leverantörsuppgifter, inte publicerad intern API-specifikation eller oberoende driftsrevision.

---

## [S08] HKM1800 — robot for intralogistics and warehouse automation

Tillverkardokumentation

Cognibotics. Utan datum. Kontrollerad 2026-10-01.

<https://www.cognibotics.com/en/products/hkm>

Används för att skilja verklig hybridkinematik från projektets generiska arm. Tillverkarens kapacitetsuppgifter är inte simulatorresultat.

---

## [S10] SICS AI: HPC Access to LUMI Supercomputer for AI and Robotics

Teknisk samarbetsbeskrivning

ENCCS. 12 september 2025. Kontrollerad 2026-10-01.

<https://enccs.se/2025/09/news/sics-hpc-access-lumi-supercomputer-ai-robotics-3/>

Dokumenterar arbete med beräkningsmiljö och GPU-träning. Fastställer inte kundernas aktuella inferensarkitektur.

---

## [S11] SICS AI investerar i NVIDIA DGX B200 från Aixia

Leverantörens pressmeddelande

Aixia Group AB. 17 juni 2025. Kontrollerad 2026-10-01.

<https://news.cision.com/se/aixia-group-ab--publ-/r/sics-ai-investerar-i-nvidia-dgx-b200-fran-aixia---accelererar-utvecklingen-av-ai-hjarna-for-nasta-ge%2Cc4165473>

Bekräftar offentliggjord DGX B200-affär med drifttjänster. Inte ett test av modeller eller belägg för var dagens maskin är placerad.

---

## [S12] MCP Server

Officiell verktygsdokumentation

Blender Lab / Blender Foundation. Löpande dokumentation. Kontrollerad 2026-10-01.

<https://www.blender.org/lab/mcp-server/>

Dokumenterar separat MCP-integration och varnar för exekvering av modellgenererad kod. Sidan anger Blender 5.1 eller senare för integrationen. Faktisk projektkompatibilitet återstår att testa.

---

---

## [S17] Making retries safe with idempotent APIs

Primär teknisk designbeskrivning

Amazon Web Services, Builders' Library. Utan datum. Kontrollerad 2026-10-01.

<https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>

Stabila begärandeidentiteter och idempotent semantik. Projektets slutsatser om fysiska effekter är egen designanalys, inte en utlovad AWS-garanti för robotar.

---

## [S18] OPC Unified Architecture

Protokolläggarens dokumentation

OPC Foundation. Löpande dokumentation. Kontrollerad 2026-10-01.

<https://opcfoundation.org/about/opc-technologies/opc-ua/>

Informationsmodell och kommunikationstjänster. En lokal signalmodell eller REST-route innebär inte implementerat OPC UA-stöd.

---

## [S19] Creating a GitHub Pages site; configuring a publishing source

Officiell publiceringsdokumentation

GitHub Docs. Löpande dokumentation. Kontrollerad 2026-10-01.

<https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site>

Pages används för statiska rapporter och nedladdningar, inte för Python eller Blender-runtime.

Kompletterande primärkälla: <https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site>

---

# Teknisk dialog om RobotOps Twin

Diskussionsunderlag för Axel Kaliff och Per-Eric Olsson

Rami Halabi

Version 1.0 · 1 oktober 2026

Oberoende tekniskt rapportpaket · Svenska

**Status: design och forskningsunderlag.** Simulatorens beskrivning som ett planerat system. Inga empiriska robotresultat eller säkerhetsgarantier hävdas.

AI-assisterad text- och publiceringsframställning.

Ej sakkunniggranskad. Inte framtagen av eller godkänd av SICS AI.

Rapporternas källor, diagram och byggsript finns i projektets repository.

---

# Innehåll

|                                                            |    |
|------------------------------------------------------------|----|
| Så använder du underlaget .....                            | 3  |
| 1. Din kärnberättelse på 90 sekunder .....                 | 3  |
| 2. Ett bra upplägg för samtalet .....                      | 3  |
| 3. Förklara arkitekturen utan att fastna i termer .....    | 4  |
| 4. Samtalsområden med Axel Kaliff .....                    | 5  |
| 5. Samtalsområden med Per-Eric Olsson .....                | 7  |
| 6. Gemensamt whiteboardfall: kvittensen försvinner .....   | 9  |
| 7. Designbeslut du bör kunna försvara .....                | 10 |
| 8. Frågor du kan få och en bra svarsriktning .....         | 10 |
| 9. Frågor om rollen som projektet hjälper dig ställa ..... | 11 |
| 10. Det du ska ha redo före en faktisk demo .....          | 12 |
| 11. Samtalsprotokoll att fylla i .....                     | 12 |
| 12. Avslutning .....                                       | 13 |
| Referenser och källbegränsningar .....                     | 13 |

## Så använder du underlaget

Det här dokumentet är ett **arbetsunderlag för teknisk dialog**, inte ett manus som ska läsas upp ord för ord. Det hjälper projektägaren att presentera RobotOps Twin, förklara sina beslut och pröva antaganden tillsammans med Axel Kaliff och Per-Eric Olsson.

Version 1.0 gäller en specificerad men ännu inte verifierad simulator. Formuleringar som "jag har byggt", "testerna visar" eller "det fungerar" får användas först när de motsvarar körbar kod och sparade resultat. Fram till dess är "jag har specificerat", "jag föreslår" och "jag vill testa" rätt ord.

Samtalet ska visa förståelse, inte bara att många källor har lästs. En bra diskussion kan sluta med att ett eget designval behöver ändras. Det är ett användbart resultat.

## 1. Din kärnberättelse på 90 sekunder

Jag läste rollen som ett uppdrag att få AI, robotcell och kundsystem att fungera tillsammans, inte bara att koppla in en modell. Därför har jag specificerat en liten integrationssimulator där ett ERP/WMS skapar plockuppdrag och Blender ersätter den fysiska världen.

Jag har medvetet inte försökt återskapa er proprietära AI. Jag skiljer på kundens avsikt, ett modellförslag, ett robotkommando och den observerade effekten. Modell och robot är utbytbara gränssnitt, men deras garantier måste vara uttryckliga.

Huvudfallet jag vill testa är att ett plock genomförs men att kvittensen försvinner. Då ska systemet inte automatiskt plocka igen, utan visa att utfallet är okänt och stämma av journal och observation. Jag vill också visa ett fall där informationen inte räcker och systemet måste stanna för granskning.

Det jag gärna vill pröva med er är vilka av dessa gränser som motsvarar verkliga problem hos er, och vad jag har förenklat för mycket.

Bakgrunden till fokus på kundintegration finns i rollannonsen. Det betyder inte att simulatören uppfyller alla anställningskrav eller återger den verkliga implementationen. [\[S01\]](#)

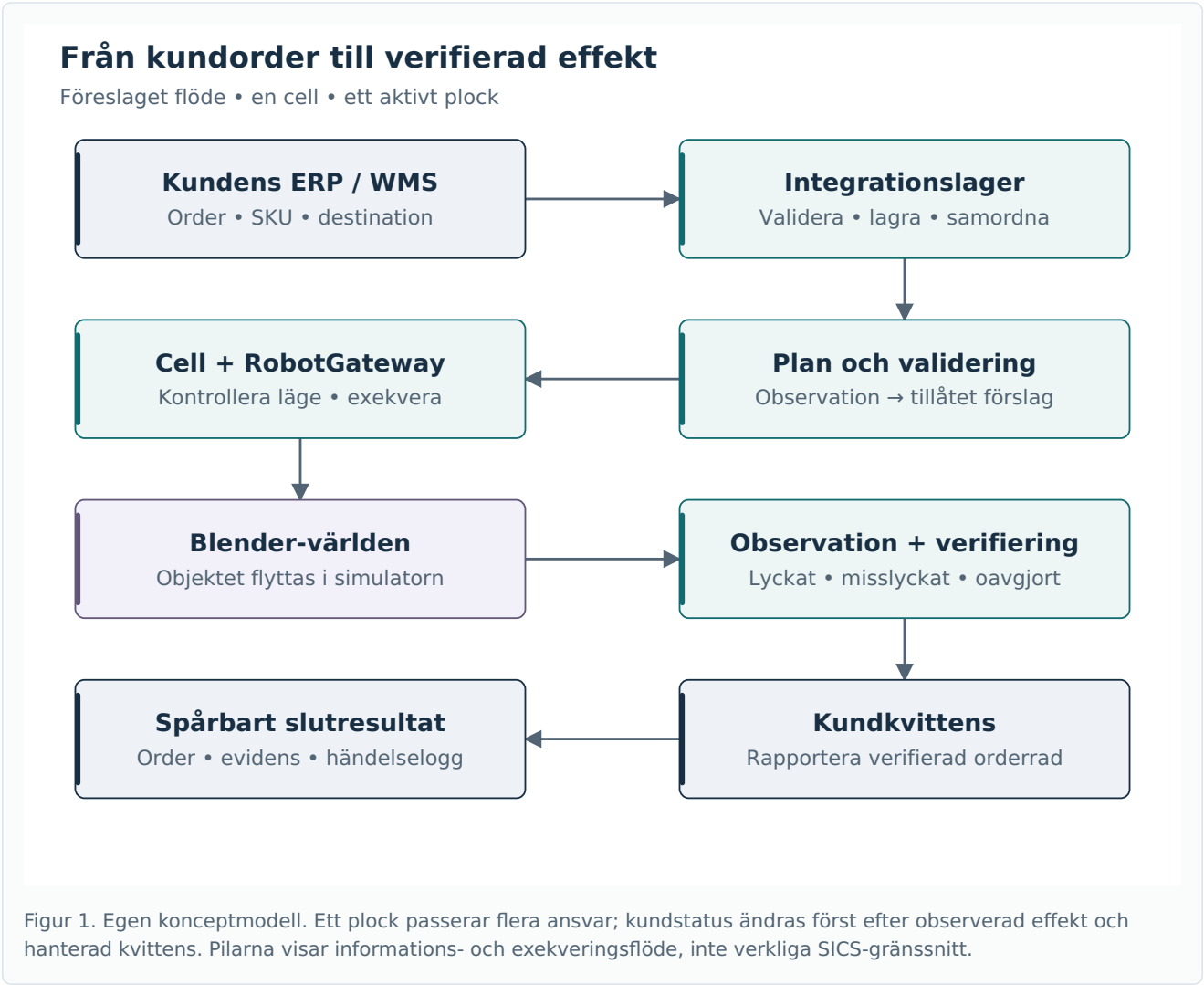
## 2. Ett bra upplägg för samtalet

| Del     | Vad du visar                    | Vad du vill lära dig    |
|---------|---------------------------------|-------------------------|
| Öppning | Syftet och en ärlig statusruta. | Är problemet rätt valt? |

| Del         | Vad du visar                                              | Vad du vill lära dig                                |
|-------------|-----------------------------------------------------------|-----------------------------------------------------|
| Arkitektur  | Kund, integration, modell, cell, Blender och verifierare. | Var går motsvarande gränser hos dem?                |
| Normalflöde | En order och dess planerade resa genom systemet.          | Vilka kvittenser och observationer behövs?          |
| Felväg      | Förlorad kvittens efter effekt.                           | Hur hanterar de osäkra eller delvisa utfall?        |
| Fördjupning | En designavvägning, inte tio verktygsnamn.                | Vad skulle de ändra och varför?                     |
| Avslutning  | Viktigaste lärdomen och nästa test.                       | Vad skulle vara ett relevant första arbetsresultat? |

Planera en kärnpresentation på ungefär tio minuter, men följ deras frågor. Det är ett förslag till disposition, inte en uppgift om mötets faktiska längd eller intervjuformat.

### 3. Förklara arkitekturen utan att fastna i termer



En möjlig förklaring är:

ERP:t vet vilken order kunden vill ha plockad. Integrationslagret håller reda på uppdraget och dess historik. Brain-komponenten föreslår en handling utifrån en observation. Validatorn kontrollerar att förslaget passar aktuell cell och aktuellt tillstånd. Robotadaptorn utför kommandot i Blender. Verifieraren bedömer resultatet innan ERP:t får en slutlig kvittens.

Förbered en mening om varje ansvar. Börja inte med alla bibliotek, processer eller klassnamn. Om de frågar om implementation går du vidare till ett konkret kontrakt, en tillståndsövergång och ett test.

### Tre skillnader som du ska kunna förklara

**Accepterat är inte utfört.** En robotcontroller kan ha tagit emot kommandot utan att ha avslutat det.

**Utfört är inte lyckat.** En rörelsesekvens kan vara färdig utan att rätt objekt hamnat rätt.

**Verifierat är inte kundsynkat.** Plocket kan vara korrekt medan uppdateringen av WMS fortfarande väntar. Då ska bara kommunikationen återförsökas, inte plocket.

Detta är resonemanget i vår design, inte påståenden om vilka fel SICS AI har i produktion.

## 4. Samtalsområden med Axel Kaliff

Axel är medförfattare till R900, och AutoGrasper-materialet beskriver separata roller för uppgift, datainsamling, återställning och felåterhämtning. Det gör dessa ämnen till relevanta, källgrundade ingångar. Det säger inte vilka frågor han kommer att ställa på en intervju. **[S05]**  
**[S06]**

### 4.1 Verifiering av faktisk uppgiftsframgång

**Säg:** "I min design är verifieraren en egen komponent. Jag vill inte att samma funktion som flyttar objektet också ska vara enda beviset för att uppgiften lyckades."

**Fråga:** "Vilken evidens brukar vara mest användbar för att skilja en avslutad rörelse från en lyckad uppgift? Hur hanterar ni ett resultat där sensorerna inte räcker för att avgöra?"

**Följ upp:** Be om ett avgränsat, icke-konfidentiellt exempel på falskt godkännande respektive falskt underkännande. Fråga om toleranserna beror på artikel, uppgift eller cell.

**Ta med dig:** Vilka observationer, tidsfönster och eftervillkor du bör modellera. Anta inte att en enda kamerabild räcker.

## 4.2 Ground truth och en begränsad observation

**Säg:** "Blender vet allt om scenen, men jag vill inte ge beslutsvägen samma tillgång. Test-oraklet får läsa facit; ordinarie verifiering ska arbeta med en begränsad observation."

**Fråga:** "Vilka gemensamma antaganden skulle ändå kunna göra mitt test missvisande? Hur skulle du konstruera ett scenario där verifieraren får fel trots att simulatorns metadata är konsekventa?"

**Följ upp:** Diskutera skymning, identitetsförväxling, gammal observation och fel kalibrering. Be inte bara om en bättre modell; försök förstå vilken informationslucka som orsakar felet.

**Ta med dig:** Minst ett testfall som inte kan lösas genom att bara läsa ett objekt-ID.

## 4.3 Återställning som en del av försöket

**Säg:** "AutoGrasper skiljer mellan normal uppgift, reset och recovery. Jag använder det som inspiration för egna tillstånd, inte som om det vore er produktionskod." [S05]

**Fråga:** "Vad brukar en automatisk reset behöva verifiera innan nästa episod startar? Vilka fel bör göra att en episod underkänns i stället för att återställas i tysthet?"

**Följ upp:** Hur undviks att misslyckade återställningar förorenar nästa försök? Ska övergången till nästa episod kräva känd objektplacering, känd robotpose eller båda?

**Ta med dig:** Ett konkret startvillkor och en loggpost som behövs för reproduktion.

## 4.4 Datainsamling och försöksdesign

**Säg:** "Jag vill spara misslyckade körningar lika noggrant som lyckade. Annars riskerar demonstrationen att bli ett urval av bra sekvenser."

**Fråga:** "Vilka metadata är lättast att glömma men viktigast när ett robotförsök senare ska förstås eller jämföras?"

**Följ upp:** Fråga om versionsspårning, kamerakalibrering, tidsstämplar, återställningsstatus och hur försöken delas upp vid utvärdering. R900 används här som inspiration för experimentdisciplin, inte som ett dataset vi har reproducerat. [S06]

**Ta med dig:** En minimal men användbar körningsmanifeststruktur.

## 4.5 Examensarbetets koppling: formulera dig försiktigt

En lämplig formulering är: "Jag såg examensarbetstiteln om Simulation In The Loop och action-success verification. Jag har inte kunnat granska hela fulltexten, så jag vill inte likställa min design med din metod. Vilken del av problemet tycker du är viktigast att skilja från en enkel efterhandskontroll?" [S04]

Det ger honom möjlighet att förklara skillnaden. Undvik att säga att examensarbetet visar att vår Blender-arkitektur fungerar eller att just hans metod har implementerats.

## 5. Samtalsområden med Per-Eric Olsson

Per-Eric presenteras av företaget som Chief AI Architect. HYPER-materialet ger en anledning att diskutera världsrepresentation, feedback och gränssnitt, men det ger inte insyn i alla tekniska detaljer. [S20] [S03]

### 5.1 Världsmodell kontra tillståndslagring

**Säg:** "Jag skiljer på simulatorns verkliga tillstånd, det observerade tillståndet, modellens interna representation och uppdragets mål. Min objektlista är ett integrationsformat; jag kallar den inte en reproduktion av er inlärd världsmodel."

**Fråga:** "På den nivå ni kan beskriva offentligt: vad ligger i modellens representation och vad ligger i vanlig systemlogik runt modellen?"

**Följ upp:** Hur hanteras tid, beständighet och osäker information? Hur märker integrationslaget att en plan bygger på gammalt tillstånd?

**Ta med dig:** En tydligare gräns mellan modellansvar och systemingenjörens ansvar, inte proprietära algoritmdetaljer.

### 5.2 Gränsen mellan uppgift och rörelse

**Säg:** "Jag har sett både beskrivningar av lednära styrning och av en separat motion-plattform. Därför har jag inte antagit att AI:n alltid returnerar en plockpunkt eller alltid en ledtrajektoria." [S03] [S07]

**Fråga:** "Vilken nivå bör en integrationsingenjör normalt tänka på: uppgiftsavsikt, greppose, bana eller tidsatta ledkommandon? Var valideras respektive nivå?"

**Följ upp:** Är samma gränssnitt relevant för alla robotar, eller finns profiler och kapabiliteter? Vem bestämmer när något ligger utanför en robots förmåga?

**Ta med dig:** Rätt abstraktionsnivå för MVP. Två möjliga kontrakt betyder inte att två fullständiga controllers måste byggas.

### 5.3 Feedback och ändrat beteende

**Säg:** "För en deterministisk baslinje kan jag återskapa logiken med samma version och indata. Om beteendet ändras under drift behöver jag veta vad som ändrats för att kunna felsöka."

**Fråga:** "Hur exponeras förändringar som är viktiga för drift: modellversion, representation, minne eller konfiguration? Vad behöver loggas för att en incident ska gå att återskapa?"

**Följ upp:** Finns en gräns mellan insamling, träning och godkänd modell för körning? Hur hanteras rollback och validering av ändrade beteenden?

**Ta med dig:** En begriplig livscykel för modell- och systemversioner. Anta inte att "kontinuerligt lärande" betyder att vikter uppdateras fritt efter varje plock.

## 5.4 Determinism, korrekthet och tidskrav

**Säg:** "Jag behandlar reproducerbarhet, rätt resultat och rätt tid som olika egenskaper. Ett system kan ge samma fel konsekvent."

**Fråga:** "När ni använder ordet deterministisk, vilken nivå syftar ni på: modellens inferens, planeringen, exekveringen eller tidsbeteendet?"

**Följ upp:** Vilka delar behöver hårda tidsgränser och vilka får vänta? Hur skiljer man en långsam modell från kötid, sensorfördröjning eller ett väntande cellvillkor?

**Ta med dig:** En faktisk tidsbudget och en tydlig terminologi, inte en abstrakt diskussion om huruvida AI "är deterministisk".

## 5.5 LLM/VLM runt den egna modellen

**Säg:** "Jag vill lägga en valfri modell bakom ett schema och låta den lämna ett förslag. Den får inte fri exekveringsrätt i Blender."

**Fråga:** "Vilka LLM/VLM-uppgifter ligger runt kärnmodellen i den här rollen? Är nyttan främst perception, tolkning av kunddata, diagnos, planering eller utvecklingsverktyg?"

**Följ upp:** Hur hanteras avvisade svar, hallucinering och otillräcklig evidens? Vad ska vara möjligt att köra utan extern modellanslutning?

**Ta med dig:** En konkret integrationsuppgift som motsvarar rollen, inte ett antagande att Codex ska ersätta deras robotbrain.

## 5.6 Compute utan att fastna i specifikationer

**Fråga:** "Vilka steg körs centralt och vilka måste fungera lokalt i cellen? Hur påverkar en förlorad nätverksförbindelse modellkörning och fortsatt drift?"

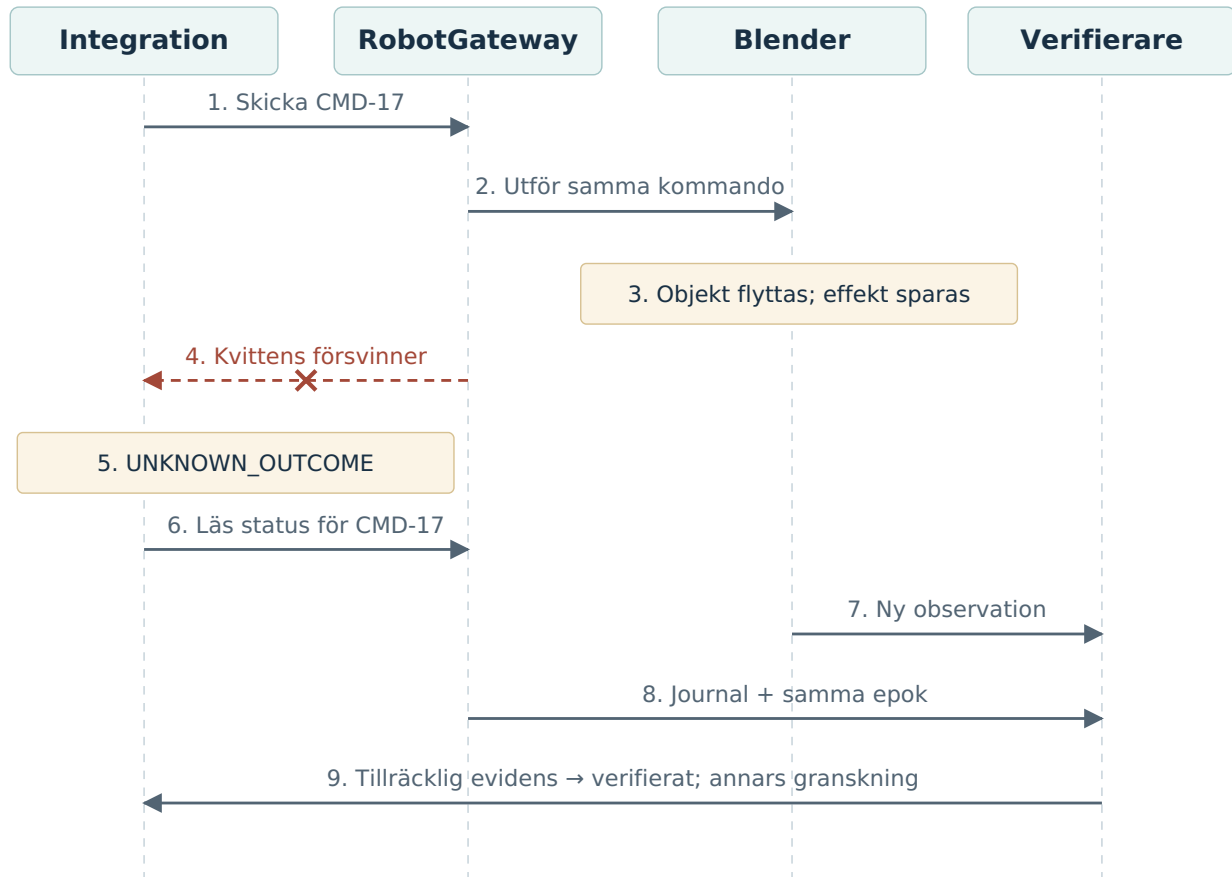
**Följ upp:** Fråga vem som äger paketering, drivrutiner, kapacitetsplanering och modellutrullning. LUMI- och DGX-källorna visar beräkningsarbete men besvarar inte rollens exakta ansvar. [\[S10\]](#) [\[S11\]](#)

**Ta med dig:** Gränsen mellan träningsinfrastruktur, inferenstjänst och cellnära integration. Du behöver inte lova att du kan administrera deras kluster utan att ha sett kraven.

## 6. Gemensamt whiteboardfall: kvittensen försvinner

### Utfört plock — förlorad kvittens

Planerat felscenario T04 • inget empiriskt testresultat



**Ingen ny PICK skickas under avstämningen.**

Figur 4. Egen sekvensmodell. Effekten uppstår före kommunikationsfelet. Integrationen begär kommandostatus och observation, inte ett nytt plock. Endast tillräcklig och konsistent evidens tillåter slutförande; annars krävs fortsatt utredning.

Beskriv först bara observationen: "Klienten fick timeout efter ett plockkommando." Be dem sedan resonera tillsammans med dig om vilka händelser som kan ligga bakom.

Ditt eget förslag:

Jag registrerar avsikten före sändning och använder ett stabilt kommandokontrakt. Efter timeout vet jag inte om effekten uteblev. Jag vill fråga efter samma kommando och ta en ny observation. Om journalen är konsistent och objektet verifieras i destinationen behöver jag inte utföra plocket igen. Om underlaget är motstridigt eller saknas ska ärendet bli synligt oavgjort.

Bra följdfrågor är: "Hur länge behåller kontrollern kommandostatus?", "Vad betyder not found efter omstart?", "Kan ett objekt ligga kvar i griparen?" och "Vad krävs för att ett nytt försök ska vara tillåtet?"

Undvik att lova exactly-once utan att definiera vilken gräns och felmodell garantin gäller för. Poängen är inte att vinna en diskussion om terminologi, utan att undvika en ogrundad fysisk sidoeffekt.

## 7. Designbeslut du bör kunna försvara

| Beslut                        | Ditt motiv                                                    | Kostnad eller begränsning                              |
|-------------------------------|---------------------------------------------------------------|--------------------------------------------------------|
| En cell och ett aktivt plock  | Gör ägarskap och felvägar begripliga.                         | Ingen flerrobotskalning demonstreras.                  |
| SQLite i första versionen     | Liten, beständig lokal miljö.                                 | Inte bevis för distribuerad hög tillgänglighet.        |
| Separat kundkvittens          | Skiljer fysisk effekt från affärssystemets uppdatering.       | Fler tillstånd och fler tester.                        |
| Deterministisk Brain-baslinje | Gör E2E-fel reproducerbara utan modellkonto.                  | Visar inte egen inlärd robotpolicy.                    |
| Begränsad Blender-runtime     | Tydliga operationer och felsökningsbarhet.                    | Mindre frihet än godtycklig agentkod.                  |
| MCP bara vid utveckling       | Skiljer kraftfulla författarverktyg från exekveringskontrakt. | Två separata arbetssätt måste dokumenteras.            |
| Verifierare och test-orakel   | Gör felaktiga slutsatser möjliga att mäta.                    | Delade simulatorantaganden kvarstår.                   |
| UNKNOWN som eget tillstånd    | Undviker att gissa vid otillräcklig evidens.                  | Kräver ett arbetssätt för granskning och återhämtning. |

Fråga gärna vilket av dessa val de först skulle ändra. Ett bra svar från dig kan vara: "Det verkar rimligt; då behöver jag ändra det här kontraktet och lägga till det här testfallet."

## 8. Frågor du kan få och en bra svarsriktning

### "Varför Blender och inte en full robotsimulator?"

"Jag optimerar första versionen för att synliggöra integrationsflödet och felvägarna. Jag gör inga påståenden om realistiska kontaktkrafter. Om frågan blir fysik, styrengrannhet eller sim-to-real behöver simulatorvalet och testmodellen omprövas."

## **"Är detta verkligen AI?"**

"Baslinjen är deterministisk planeringslogik. Den är medvetet inte ett forskningsbidrag inom robotinlärning. AI-adaptern är valfri och får lämna strukturerade förslag. Det jag vill demonstrera är systemansvaret runt en sådan komponent."

## **"Hur vet du att plocket lyckades?"**

"Jag behöver en definierad framgångsregel och observation efter handling. I test kan ett separat orakel kontrollera om verifieraren gjorde rätt. I verklig utrustning måste sensor- och verifieringskedjan valideras på nytt."

## **"Varför inte bara retry?"**

"För att samma timeout kan betyda att ingenting hände eller att handlingen redan utfördes. Jag börjar med att bestämma vad mottagaren garanterar och hur utfallet kan stämmas av."

## **"Vad händer om modellen returnerar ett giltigt men felaktigt kommando?"**

"Schema är bara första kontrollen. Förslaget måste också passa aktuell observation, objektidentitet, celltillstånd, kalibrering och tillåtna operationer. Även efter det kan uppgiften misslyckas, så resultatet behöver verifieras."

## **"Hur mycket av detta gjorde AI åt dig?"**

"Jag använder AI för utkast och implementation, men ansvarar för att förstå och granska ändringarna. Jag ska kunna förklara den kod jag visar, vilka antaganden den gör och vilka tester som faktiskt har körts."

Visa gärna en konkret diff, ett test som först misslyckades och hur du verifierade ändringen. Säg inte att du har gjort detta om det ännu inte finns.

## **"Kan vi byta Blender mot vår robot direkt?"**

"Inte utan nytt integrationsarbete. Affärsgränsen är avsiktligt separerad, men adapter, tidskrav, kalibrering, säkerhetsansvar och hårdvarutester måste specificeras för er miljö."

---

## **9. Frågor om rollen som projektet hjälper dig ställa**

Be om ett konkret exempel på ett första uppdrag: vilket kundflöde som ska fungera, vilka system som finns och vem som accepterar leveransen. Det är mer användbart än en allmän fråga om teknikstack.

Fråga också hur ansvaret delas mellan modellutveckling, integration, robotleverantör och drift. Vem prioriterar när en kundanpassning konkurrerar med plattformarbete? Vilken hjälp finns inom områden som är nya för dig? Vilka delar förväntas du äga själv efter tre månader?

En bra avslutande rollfråga är:

”Vilken av de här integrationsfrågorna skulle ni vilja att jag kan lösa självständigt först, och hur skulle ni avgöra att lösningen är tillräckligt bra för kund?”

Detta är frågor om arbetsinnehåll och leveransförväntningar, inte ett löfte från dig om kompetens som du ännu inte kan belägga.

## 10. Det du ska ha redo före en faktisk demo

Förbered en ärlig statusruta med implementerat, simulerat, testat och återstående. Ha versionsnumret synligt. Visa ett litet dataexempel, en tillståndsovergång och minst ett relevant feltest. Rensa bort nycklar och privata kunddata från skärmen och terminalhistoriken som du visar.

Om programmet fungerar: skapa ordern live, följ ID:t genom systemet och visa Blender-effekten. Visa sedan kvittensbortfall och ett oavgjort fall. Om programmet inte är färdigt: använd diagrammen och säg att det är designgranskning. En inspelad sekvens ska tydligt presenteras som inspelad.

Ha en reservversion av rapporterna som PDF. Börja inte installera nya drivrutiner eller flytta runtime till en okänd miljö precis före presentationen. Det är en praktisk rekommendation för denna demo, inte en uppgift om SICS interna rutiner.

## 11. Samtalsprotokoll att fylla i

| Fråga eller antagande                    | Vad de faktiskt sade | Designkonsekvens | Nästa verifiering |
|------------------------------------------|----------------------|------------------|-------------------|
| Var slutar modellens ansvar?             |                      |                  |                   |
| Vad räknas som lyckat plock?             |                      |                  |                   |
| Vilka fel är vanligast?                  |                      |                  |                   |
| Vad lagras vid en omstart?               |                      |                  |                   |
| Vilka observationer är tillförlitliga?   |                      |                  |                   |
| Vilka protokoll används i relevant cell? |                      |                  |                   |
| Vad bör MVP inte försöka göra?           |                      |                  |                   |

| Fråga eller antagande                      | Vad de faktiskt sade | Designkonsekvens | Nästa verifiering |
|--------------------------------------------|----------------------|------------------|-------------------|
| Vad är en rimlig första leverans i rollen? |                      |                  |                   |

Anteckna deras faktiska formuleringar och skilj dem från din tolkning. Publicera inte nya mötesuppgifter eller interna detaljer i det offentliga repot utan att först stämma av vad som får delas.

## 12. Avslutning

En möjlig sammanfattning efter samtalet är:

Jag ville använda projektet för att göra mina antaganden synliga, inte för att låtsas känna er implementation. Det viktigaste jag tar med mig är var ni sätter gränsen mellan modell, integration och robot, och vilken evidens som behövs för att ett uppdrag ska räknas som lyckat. Nästa steg för mig är att pröva de antagandena med ett litet reproducerbart test.

**Det starkaste du kan visa är inte att du redan kan allt. Det är att du kan avgränsa ett problem, formulera ett testbart kontrakt, förklara en avvägning och ändra uppfattning när bättre information kommer fram.**

## Referenser och källbegränsningar

Referenserna avser avgränsade sakuppgifter. Diagram, kontrakt och testplan är egna förslag. Gemensamt kontrolldatum: 1 oktober 2026.

### [S01] Systemingenjör — annons 4464336388

Arbetsgivarens rollbeskrivning

Superintelligence Computing Systems. Utan säkert publiceringsdatum. Kontrollerad 2026-10-01.

<https://se.linkedin.com/jobs/view/systemingenj%C3%B6r-at-superintelligence-computing-systems-4464336388>

Offentligt sökindexerad annons kontrollerad. Används som kravunderlag; inte som bevis för intern teknikstack. Direktsidan kunde inte återläsas konsekvent.

### [S03] Foundation AGI model for Industrial Robots — HYPER: reporting

Självrapporterat projektarbete

SICS AI, deltagarrapport hos Europeiska kommissionen/CORDIS. Uppdaterad 1 juni 2026. Kontrollerad 2026-10-01.

<https://cordis.europa.eu/project/id/101218436/reporting>

Projekt-ID 101218436. Beskrivningen av modell och robotkontroll är deltagarens redovisning, inte oberoende reproduktion. DOI för projektet: 10.3030/101218436.

---

## [S04] Simulation In The Loop: Real-Time Robot Action Success Verification with a Digital Twin

Examensarbete — bibliografisk uppgift

Axel Kaliff, KTH. 2025. Kontrollerad 2026-10-01.

<https://kth.diva-portal.org/smash/record.jsf?pid=diva2%3A1988006>

Titel och författarsamband stöds av examinatorns offentligt indexerade KTH-sida. DiVA-fulltexten kunde inte hämtas vid denna kontroll. Inga detaljerade metod- eller resultatpåståenden återges.

Kompletterande primärkälla: <https://www.csc.kth.se/~fpokorny/>

---

## [S05] Project Autograsper: Custom Grasper Development Guide

Öppen primärdokumentation

CloudGripper-projektet. Löpande repositorydokumentation. Kontrollerad 2026-10-01.

<https://github.com/cloudgripper/autograsper/blob/main/README.md>

README granskad via GitHub. Git-blob: 3cb1da256bc5147ef52a56327da476f110c4e6c2. Forskningsverktyg; inte SICS AI:s produktkod. Ingen kod eller fjärrrobotkörning har återanvänts i rapportpaketet.

---

## [S06] R900: Understanding the Cost-Effectiveness of Random Exploration from 900 Hours of Robotic Data Collection

Forskningspreprint

Shutong Jin, Axel Kaliff, Ruiyu Wang, Muhammad Zahid och Florian T. Pokorny. arXiv v2, 17 september 2025. Kontrollerad 2026-10-01.

<https://arxiv.org/abs/2503.23571>

DOI: 10.48550/arXiv.2503.23571. Används för datainsamling, episodhantering och återställning som designinspiration. Projektet reproducerar inte experimenten eller resultaten.

---

## [S07] Nowaste Logistics Advances Automation with Cognibotics Robot Picking Cell

Leverantörens installationsbeskrivning

Cognibotics. 5 januari 2026. Kontrollerad 2026-10-01.

<https://www.cognibotics.com/en/articles/nowaste-logistics-advances-automation-with-cognibotics-robot-picking-cell>

Beskriver andra cellen, AutoStore, SICS AI-vision och Juliet & Romeo. Leverantörsuppgifter, inte publicerad intern API-specifikation eller oberoende driftsrevision.

---

## [S10] SICS AI: HPC Access to LUMI Supercomputer for AI and Robotics

Teknisk samarbetsbeskrivning

ENCCS. 12 september 2025. Kontrollerad 2026-10-01.

<https://enccs.se/2025/09/news/sics-hpc-access-lumi-supercomputer-ai-robotics-3/>

Dokumenterar arbete med beräkningsmiljö och GPU-träning. Fastställer inte kundernas aktuella inferensarkitektur.

---

## [S11] SICS AI investerar i NVIDIA DGX B200 från Aixa

Leverantörens pressmeddelande

Aixa Group AB. 17 juni 2025. Kontrollerad 2026-10-01.

<https://news.cision.com/se/aixa-group-ab--publ-/r/sics-ai-investerar-i-nvidia-dgx-b200-fran-aixa---accelererar-utvecklingen-av-ai-hjarna-for-nasta-ge%2Cc4165473>

Bekräftar offentliggjord DGX B200-affär med drifttjänster. Inte ett test av modeller eller belägg för var dagens maskin är placerad.

---

---

## [S20] About Us

Företagets egen presentation

SICS AI. Utan datum. Kontrollerad 2026-10-01.

<https://sics.ai/about/>

Används endast för den uppgivna rollen Chief AI Architect för Per-Eric Olsson. Övriga karriär- och AGI-påståenden övertas inte som verifierade meriter.

---